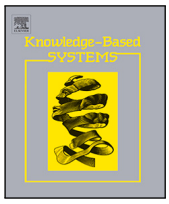
Knowledge-Based Systems 302 (2024) 112402



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





Two-step dynamic obstacle avoidance

Fabian Hart [a], Martin Waltz [a,*], Ostap Okhrin [a,b]

[a] Technische Universität Dresden, Chair of Econometrics and Statistics, esp. in the Transport Sector, Dresden, 01062, Wuerzburger Str. 35, Germany
[b] Center for Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Dresden/Leipzig, Germany

ARTICLE INFO

Keywords:
Deep reinforcement learning
Supervised learning
Dynamic obstacle avoidance
Local path planning

ABSTRACT

Dynamic obstacle avoidance (DOA) is a fundamental challenge for any autonomous vehicle, independent of whether it operates in sea, air, or land. This paper proposes a two-step architecture for handling DOA tasks by combining supervised and reinforcement learning (RL). In the first step, we introduce a data-driven approach to estimate the collision risk (CR) of an obstacle using a recurrent neural network, which is trained in a supervised fashion and offers robustness to non-linear obstacle movements. In the second step, we include these CR estimates into the observation space of an RL agent to increase its situational awareness. We illustrate the power of our two-step approach by training different RL agents in a challenging environment that requires to navigate amid multiple obstacles. The non-linear movements of obstacles are exemplarily modeled based on stochastic processes and periodic patterns, although our architecture is suitable for any obstacle dynamics. The experiments reveal that integrating our CR metrics into the observation space doubles the performance in terms of reward, which is equivalent to halving the number of collisions in the considered environment. We also perform a generalization experiment to validate the proposal in an RL environment based on maritime traffic and real-world vessel trajectory data. Furthermore, we show that the architecture's performance improvement is independent of the applied RL algorithm.

1. Introduction

Autonomous vehicles represent a promising avenue for enhancing safety and optimizing energy and fuel efficiency within traffic systems, making them a vital element of future sustainable transportation networks [1]. While the general public often fixates on self-driving cars [2], the potential benefits of autonomous vehicles extend to other modes of transportation, such as autonomous vessels [3] and aircraft systems [4]. Regardless of whether a vehicle is under human or autonomous control, the paramount concern in all traffic operations remains ensuring safety by preventing collisions with other vehicles and stationary obstacles [5,6].

We focus on entities operating in non-lane-based traffic environments, irrespective of whether they steer a vessel, fly an aircraft, command a drone, or control any other vehicle. These entities are referred to as *agents*. Furthermore, we will call traffic participants close to the agent *obstacles*. Generally, an agent's objective is to reach a specific destination safely. A real-world system typically relies on a perception-planning-control (PPC) modularization [7,8]. The perception module includes sensor systems such as LiDAR, radar, or camera to localize the agent and relevant obstacles in the environment accurately [9]. Moreover, considering the noise and uncertainties associated with real-world sensors, advanced state estimation techniques are often applied [10]. The planning module processes the information from the perception module and generates a safe and feasible local path in the form of a set of waypoints [11]. Crucially, the path planning should consider the maneuverability of both the agent and the obstacles. Lastly, the control module translates high-level commands derived from the local path into low-level actuator control commands [12].

In this paper, we consider the task of dynamic obstacle avoidance (DOA, [13]), which we define as the challenge of executing real-time collision avoidance maneuvers while facing obstacles with non-zero velocity vectors. Following the formalism of [4], we consider a distributed setting where no communication between traffic participants takes place, which is a practically occurring situation across traffic domains. DOA can be attributed to local path planning in the PPC modularization, and we review popular methods for DOA in Section 2. In the following, we deviate from traditional methods and consider specifically the emerging method of deep reinforcement learning (DRL, [14]). DRL tackles sequential decision tasks, in which an agent interacts with an environment and learns based on trial-and-error. Over the last years, the methodology has led to remarkable successes in various application domains [15–19]. In our DOA task, the agent receives an observation of the traffic situation, takes a steering or acceleration action, and obtains a numerical reward signal that should be maximized.

* Corresponding author.
E-mail addresses: fabian.hart@tu-dresden.de (F. Hart), martin.waltz@tu-dresden.de (M. Waltz), ostap.okhrin@tu-dresden.de (O. Okhrin).

https://doi.org/10.1016/j.knosys.2024.112402
Received 18 January 2024; Received in revised form 16 August 2024; Accepted 17 August 2024
Available online 22 August 2024
0950-7051/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Several contributions emphasize the strong dependence of the reinforcement learning (RL) algorithms' performances on the precise selection of the features of the observation vector [20–22]. These features build the basis for the agent's decision-making and should contain relevant information to handle the task at hand successfully. In DOA tasks, the observation features frequently consist of the position and velocity of the agent and nearby obstacles [23–25]. In real-world situations, we acknowledge that certain rule sets, like the Convention on the International Regulations for Preventing Collisions at Sea, explicitly require estimating collision risk (CR) with other traffic participants (Rule 7, [26]). Leveraging these considerations, we aim to additionally include CR metrics into the observation of the DRL agent.

We emphasize that a similar approach has been pursued by some prior works [27–29]. However, these studies share the limitation of assuming a *linear motion* of nearby obstacles during the CR computation. For instance, popular CR measures based on the closest point of approach (CPA, [30]) like the distance between the vehicles at the CPA, denoted d^{CPA}, and the time until reaching the CPA, denoted t^{CPA}, are calculated assuming constant course and speed of the obstacle. While these widely used metrics offer great interpretability and constitute an essential tool for real-world captains and pilots, future turns or accelerations of the obstacles are not considered, leaving a significant risk of misinterpreting a given scenario.

Addressing this challenge, we propose to merge traditional highly interpretable CPA-based measures with a supervised learning approach to CR estimation to construct a comprehensive DRL agent for DOA tasks. In particular, our contributions are as follows:

- We propose a *data-driven* approach to generate an estimate of the d^{CPA} and t^{CPA} with obstacles. More precisely, we train an LSTM network [31] to estimate the future trajectories of obstacles in a supervised fashion [32], and compute the CR metrics subsequently. This procedure can generalize across situations and is robust to non-linear obstacle behavior.
- We augment the *observation space* of an RL algorithm to include the newly estimated CR metrics, enabling the agent to assess and solve various encounter situations robustly. Moreover, this proposal can be seen in context to recent advances from the robotics domain [33] since our CR estimator acts as an additional observation processing module.
- We showcase the potential of our *two-step architecture* (Step 1: CR estimation, Step 2: DRL) for handling DOA tasks, thereby underscoring its suitability for implementation inside a local path planning unit of a PPC modularization. More precisely, we define a traffic-type independent environment where an agent needs to avoid dynamic obstacles and quantify the advantage in terms of the reward difference of the proposal in comparison to a 'conventional' DRL agent for obstacle avoidance.
- Using real-world vessel trajectory data, we demonstrate that the proposed two-step architecture effectively adapts to this new environment, highlighting the architecture's robustness and generalizability. This study underscores the potential of our approach for diverse applications beyond the initial traffic-type independent environment.

The paper is structured as follows: Section 2 reviews related work, while Section 3 provides a concise and formal problem description of the considered DOA task. Section 4 describes the newly proposed method, while Section 5 provides additional implementation details. Section 6 shows the experiments, and Section 7 displays the generalization study to maritime traffic. Section 8 discusses limitations and outlines areas for future work. Section 9 concludes this article. The source code for this paper is publicly available at https://github.com/MarWaltz/Two-Step-DOA.

2. Related work

2.1. Dynamic obstacle avoidance: Method overview

DOA is a problem originating from the robotics literature with a longstanding history of seminal contributions; see [34,35]. Key methods include the artificial potential field (APF) method introduced by [36], which uses virtual forces to push robots away from obstacles. The vector field histogram method by [37] offers a fast and efficient way to navigate dynamic environments by creating a polar histogram of obstacle densities. Another significant contribution is the dynamic window approach (DWA) proposed by [38], which integrates the robot's dynamics into the obstacle avoidance process, enabling more realistic and feasible maneuvers. Furthermore, the curvature-velocity method introduced by [39] solves a constrained optimization problem to explicitly trade off speed, safety, and goal-directness to enable fast and smooth robot navigation. [40] introduced velocity obstacles (VO), which defines a set of velocities that would result in a collision with an obstacle, allowing the robot to choose a velocity that avoids these dangerous trajectories. Further influential approaches to DOA for mobile robots include ant colony optimization [41], sampling-based approaches like rapidly-exploring random trees [42], or particle swarm optimization [43].

Other traffic domains have adapted these methods to suit their individual needs. For instance, in the maritime literature, [44,45] have adjusted the APF method for DOA of vessels, while [46] integrate maritime traffic rules into the VO approach by [40]. In addition, [47] outline a path planning strategy for autonomous ships using the DWA. In the aviation community, [48] use a three-dimensional VO approach to control unmanned aerial vehicles, and [13] employ an APF approach to enable micro aerial vehicles to dodge fast-moving objects. For further applications and information on conventional DOA methods, we refer to the comprehensive reviews by [4,5] for the aviation literature, [6,49] for autonomous vessels, and [50,51] for mobile robots.

2.2. Dynamic obstacle avoidance: Reinforcement learning

The method employed in this paper, DRL, has also been successfully applied to DOA tasks across traffic domains. For instance, [52] propose a physics-informed DRL model for resolving aircraft conflicts, while [24] use a multi-agent RL approach to control aircraft in high-density en route airspaces. [53] introduce a recurrent DRL approach to autonomously navigate unmanned aerial vehicles (UAVs) in complex three-dimensional environments. Additionally, [54] apply DRL with graph neural networks to resolve conflicts among UAVs. In the maritime domain, [55,56] develop DRL approaches that explicitly consider the CR posed by nearby vessels and adhere to maritime traffic rules. [28] integrate a priority sampling mechanism into a DRL framework to facilitate vessel path planning while accounting for dynamic obstacles. In addition, [57] demonstrate a real-world application of DRL to avoid collisions involving three self-propelled ships autonomously.

Concerning mobile robot navigation, [23] outline a recurrent DRL method to navigate up to 20 agents jointly in an environment while performing DOA. [58] combine Q-Learning [59] with a neural network-based trajectory planner to let a robot avoid obstacles in a virtual and a real-world environment. In addition, [60] use the Double DQN of [61] for a mobile robot DOA task, which is evaluated on a physical robot platform. Further contributions to DOA via DRL in non-lane-based traffic scenarios include [23,62,63], and [64].

2.3. Collision risk estimation

DOA involves assessing CR with nearby obstacles to determine appropriate avoidance maneuvers. Methods for quantifying CR in non-lane-based traffic vary widely, focusing on geometric considerations of current traffic states, defining safety zones, and uncertainty in future

states; see the reviews of [65,66]. Key contributions to the aviation literature include [67], who estimate the probability of conflict (PC) between vehicles based on trajectory predictions and associated prediction error, assuming a Gaussian distribution. [68] adapt [67] to better handle vehicle maneuvers and non-linear behaviors, while [69] outline a Monte Carlo method for PC computation. More recently, [70] propose a probabilistic model to evaluate CR between an intruding drone and commercial aircraft. [71] derive rapid methods for collision probability estimation for three types of collision zones involving small unmanned aircraft systems. Finally, [72] present a unified mathematical framework for collision probability estimation in the air traffic literature.

In maritime contexts, [73] introduce a safe zone around vessels called ship domain, which is empirically investigated using real-world vessel data in [74]. Moreover, [75] detail recent developments and discuss several modifications of the ship domain concept. In addition, [76] quantify CR using an exponential relationship with the CPA metrics of [30], and [77] model the maximum of the CPA metrics of nearby vessels with a truncated gamma distribution. In recent years, [78] build a support vector machine classifier to model the probability of a vessel collision while accounting for the consequences of a potential collision. [79] estimate the ship collision probability leveraging the cross-entropy method [80], and integrate the estimates in the probabilistic sampling-based model predictive control approach of [81] to perform a DOA task. Lastly, [82] outline a multi-criteria framework for real-time CR assessment that is validated in Portuguese continental coastal waters.

3. Problem description

We consider a two-dimensional DOA task detached from a specific transportation context. We define a point-mass agent with a constant longitudinal speed for simplification while the lateral acceleration can be controlled. Since this study aims at a realistic setting with non-linearly moving obstacles, we generically define an obstacle trajectory as:

$$\vec{p}_t = \vec{p}_{t,\text{lin}} + \vec{p}_{t,\text{non}}, \tag{1}$$

where $\vec{p}_t = (x_t, y_t)^\top \in \mathbb{R}^2$ is the position of the obstacle at time step t. The terms $\vec{p}_{t,\text{lin}}$ and $\vec{p}_{t,\text{non}}$ denote the linear and non-linear components of the obstacle trajectory, respectively. More precisely, the linear part is defined as:

$$\vec{p}_{t,\text{lin}} = \vec{p}_0 + \dot{\vec{p}}\, t, \tag{2}$$

with the initial obstacle position $\vec{p}_0 = (x_0, y_0)^\top \in \mathbb{R}^2$ and the constant, obstacle-dependent velocity $\dot{\vec{p}} = (\dot{x}, \dot{y})^\top \in \mathbb{R}^2$. Eq. (2) ensures that the obstacle moves in a specific direction. At the same time, the non-linear part $\vec{p}_{t,\text{non}}$ can capture course deviations due to environmental effects or the obstacle's own avoidance maneuvers. Importantly, different practical behaviors of obstacles can be simulated via different specifications of $\vec{p}_{t,\text{non}}$. Throughout this study, we exemplarily consider two different kinds of non-linear obstacle movements: *stochastic* and *periodic* trajectories.

Stochastic trajectories. The first non-linear specification is based on a stochastic two-dimensional auto-regressive process [83]:

$$A_{t+1} = \phi_{\text{stoch}} A_t + u_{t,\text{stoch}}, \quad \text{where} \quad u_{t,\text{stoch}} \sim \mathcal{N}_2\left[\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}\sigma^2_{\text{stoch}} & 0\\ 0 & \sigma^2_{\text{stoch}}\end{pmatrix}\right], \tag{3}$$

with initial value $A_0 = (0,0)^\top$, auto-regressive parameter ϕ_{stoch}, and variance σ^2_{stoch} of the centered noise stemming from the multivariate normal distribution. Addressing the noisy nature of the process, we perform an exponential smoothing of (3):

$$A_{t,\text{smooth}} = \beta_{\text{stoch}} A_t + (1-\beta_{\text{stoch}}) A_{t-1,\text{smooth}}, \quad \text{where} \quad A_{0,\text{smooth}} = 0, \tag{4}$$

with the smoothing factor $\beta_{\text{stoch}} = 0.2$. Moreover, to account for different obstacle velocities, we scale the stochastic trajectory:

$$\vec{p}^{\,(1)}_{t,\text{non}} = \frac{\|\dot{\vec{p}}\|}{v_{\max}} A_{t,\text{smooth}}, \tag{5}$$

with maximum obstacle speed $v_{\max}$.

Periodic trajectories. The second obstacle behavior considered in this study incorporates a sinus pattern with additional noise:

$$\vec{p}^{\,(2)}_{t,\text{non}} = \begin{pmatrix}\dot{y}\\ -\dot{x}\end{pmatrix} A_{\sin} \sin\left(\frac{2\pi t}{T_{\sin}}\right) + u_{t,\sin}, \quad \text{where} \quad u_{t,\sin} \sim \mathcal{N}_2\left[\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}\sigma^2_{\sin} & 0\\ 0 & \sigma^2_{\sin}\end{pmatrix}\right], \tag{6}$$

with amplitude $A_{\sin}$, period $T_{\sin}$, and variance $\sigma^2_{\sin}$.

Figs. 3 and 4 visualize examples of the stochastic and periodic trajectories, respectively, in blue color. Throughout this study, we use a simulation step size of $\Delta t = 5$ s for simulating obstacle and agent movement. Crucially, to train a *CR estimator* that estimates CR metrics for each obstacle and for each time step, we impose the following assumption:

Assumption 1 (***Behavior stability***)**.** The behavior of the obstacles used to train the collision risk estimator is similar to the obstacle behavior in the simulation environment.

4. Proposed method

First, we provide an overview of the essential background in supervised and reinforcement learning in Sections 4.1 and 4.2, respectively. On this basis, we present our proposed two-step method for DOA in Section 4.3.

4.1. Supervised learning

Supervised learning is the most common form of machine learning and describes the task of learning a mapping from potentially high-dimensional input samples to corresponding target vectors based on labeled data [32,84]. Paired with (deep) neural networks as function approximators, supervised learning approaches have shown remarkable successes in various real-world problems such as speech recognition [85] or object detection [86]. In this study, we consider the regression task of having data pairs $D = \{(\mathbf{X}_i, \mathbf{Y}_i)\}_{i=1}^N$, where $\mathbf{X}_i \in \mathbb{R}^n$, $\mathbf{Y}_i \in \mathbb{R}^m$, and learning the parameter vector θ of a neural network $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ by minimizing the mean squared error $MSE(D;\theta) = N^{-1} \sum_{i=1}^N \left[f_\theta(\mathbf{X}_i) - \mathbf{Y}_i\right]^\top \left[f_\theta(\mathbf{X}_i) - \mathbf{Y}_i\right]$ with the Adam optimizer [87].

4.2. Reinforcement learning

RL aims at solving sequential decision tasks in which an agent interacts with an environment under the objective to maximize a numerical reward signal [14]. Formally, we consider Markov Decision Processes (MDP, [88]) consisting of a state space $\mathcal{S}$, an action space $\mathcal{A}$, an initial state distribution $T_0 : \mathcal{S} \to [0,1]$, a state transition probability distribution $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0,1]$, a reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, and a discount factor $\gamma \in [0,1)$. At each time step t, the agent receives a state information $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}$, gets an instantaneous reward R_{t+1}, and transitions based on the environmental dynamics $\mathcal{P}$ to the next state $S_{t+1} \in \mathcal{S}$. In practical applications, the full-state information is rarely available due to sensor limitations, delays, noise, or other disturbances. Addressing this issue, a generalization called Partially Observable Markov Decision Processes (POMDP, [89])

introduces two additional components: the observation space $\mathcal{O}$ and the observation function $\mathcal{Z} : S \times \mathcal{A} \times \mathcal{O} \to [0,1]$. In a POMDP, the agent does not receive the new state S_{t+1} directly, but instead an observation $O_{t+1} \in \mathcal{O}$, which is generated with probability $P(O_{t+1}|S_{t+1}, A_t)$ by the observation function $\mathcal{Z}$. Consequently, a POMDP is a Hidden Markov Model with actions and the observations are used for learning. In the following, we use capital notation, e.g., S_t, to indicate random variables and small notation, e.g., s_t or s, to describe their realizations.

Objective of the agent in the MDP scenario is to optimize for a policy $\pi : S \times \mathcal{A} \to [0,1]$, a mapping from states to probability distributions over actions, that maximizes the expected return, which is the expected discounted cumulative reward, from the start state: $E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{k+1}|S_0\right]$. Common practice is the definition of action value functions $Q^\pi(s,a)$, which are the expected return when starting in state s, taking action a, and following policy π afterwards: $Q^\pi(s,a) = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|S_t = s, A_t = a\right]$. Crucially, in an MDP there is a deterministic optimal policy $\pi^*(s) = \text{argmax}_{a\in\mathcal{A}} Q^*(s,a)$ if the state space is finite or countably infinite [88]. The policy π^* is connected with optimal action-values $Q^*(s,a) = \max_\pi Q^\pi(s,a)$. The general control objective of RL is to find or approximate π^*.

4.3. Two-step method

We visualize the proposed two-step method for DOA in Fig. 1. The approach is composed of two distinct steps, each contributing to the overall decision-making process. In the first step, a supervised learning module predicts the future trajectories of relevant obstacles while accounting for possible non-linear motions. These trajectory predictions are used to generate estimates of the d^{CPA} and t^{CPA}, which are informative CR metrics that quantify the criticality of specific obstacles. In the second step, these CR metrics are concatenated with the remaining features from the RL environment to build a comprehensive observation vector for the RL agent. The agent iteratively interacts with the environment, adjusting its actions to maximize reward and optimize the DOA policy. Through this process, the RL agent learns to make informed, context-sensitive decisions that enhance the safety and efficiency of operations within dynamic environments.

5. Implementation details

This section provides details on the implementation of the method outlined in Section 4 to solve the problem described in Section 3.

5.1. Trajectory prediction

The input features $\mathbf{X}$ of the supervised learning module contain information about $h = 10$ previous positions of the obstacle $(\vec{p}_{t-9}, \dots, \vec{p}_t)^\top$, from which the neural network should approximate the next position $\vec{p}_{t+1}$. However, we emphasize that we build the first differences of $(\vec{p}_{t-9}, \dots, \vec{p}_t)^\top$ before inputting them into the network, as we have empirically found that estimating relative rather than absolute positions is significantly more stable. We construct the neural network f_θ with an LSTM-layer [31] of 64 hidden units. The output is forwarded through two linear layers with 64 neurons to yield output in $\mathbb{R}^2$. To generate training data, we repeatedly sample trajectories according to the obstacle dynamics as given in (5) and (6) for the stochastic and periodic obstacle movements, respectively. The training progress of the network is depicted in Fig. 2 for both defined non-linear obstacle movements.

After the training is completed, we perform subsequent one-step-ahead forecasts to predict an obstacle's *entire trajectory* for a time horizon $T = 100$. More precisely, we always take the last ten positions, possibly including predicted ones, to predict the following position iteratively. To illustrate the training results, we provide a representative example for a predicted trajectory in Fig. 3 for the stochastic obstacle movement and Fig. 4 for the periodic movement. The blue dots represent the ground truth trajectory, from which only the first ten dots, shown in green, are observed. These observations are used to iteratively compute the predicted trajectory, which is shown in orange. Since we assume that not all previous positions of an obstacle are observable, we predict past positions as well. These past obstacle positions are later used to estimate if the agent already passed and obstacle; see Section 5.2. Since the obstacle trajectories defined in (5) and (6) are symmetric, we can use the same model to predict past positions.

To quantitatively evaluate the prediction accuracy, we compute the root mean squared error (RMSE) between predicted and real trajectories for both obstacle movements, as shown in Fig. 5, with different quantiles. We can see that the prediction accuracy is relatively high for short-term predictions, but expectedly increases with increased forecasting time.

5.2. Collision risk estimation

After predicting the trajectory of an obstacle for a time horizon $T = 100$, which equals a forecast time of 500 s based on a simulation step size of $\Delta t = 5$ s, we can estimate the CPA by numerically finding the minimum between the predicted obstacle and the predicted agent trajectory. For the agent, we thereby assume a linear movement based on its current velocity vector. An example is illustrated in Fig. 6 for a periodic movement of the obstacle, where the blue curve represents the estimated distance between the agent and obstacle based on these predictions. Naturally, these distance estimates exhibit the periodic oscillations of the obstacle trajectory. To reduce the variance in the estimation of d^{CPA} and t^{CPA} caused by this pattern, we apply a symmetric moving average filter on the estimated distance:

$$\hat{d}_{j,\text{smooth}} = \frac{1}{2n+1} \sum_{j=-n}^{n} \hat{d}_j,$$

where $n = 10$. The variables $\hat{d}_j$ and $\hat{d}_{j,\text{smooth}}$ denote the distance estimates between agent and obstacle at step j before and after smoothing, respectively. The resulting estimates for d^{CPA} and t^{CPA} using the smoothed distance are depicted in orange color in Fig. 6. Note that negative values for the t^{CPA} are possible if the agent already passed the obstacle.

5.3. Recurrent reinforcement learning

As described in Section 4.2, only observations o_t rather than full states s_t are available in the POMDP case. One popular approach to handle this scenario is the construction of belief states, which are distributions over the real states the agent might be in, given the observation so far. This approach requires a model of the environment and is computationally demanding [90]. An alternative approach might be to stack past observations together [91], but this quickly increases the input dimension and limits the agent to a fixed number of frames. Finally, a further approach is to incorporate recurrency into the function approximators of model-free algorithms, which was shown to yield strong performances [92]. The recurrency enriches the agent's decision making by extracting information from past observations, potentially yielding an improved ability to solve problems without access to the complete state vector.

In this study, we follow the latter approach and use the LSTM-TD3 algorithm of [93], which adds LSTM layers to the off-policy, actor-critic TD3 algorithm of [94]. Crucially, the LSTM-TD3 uses a deterministic policy μ and defines the past history h_t^l of length l at time step t defined as:

$$h_t^l = \begin{cases} o_{t-l}, \dots, o_{t-1} & \text{if} \quad l, t \geq 1. \\ o_0 & \text{else.} \end{cases}$$

The zero-valued dummy observation o_0 is of the same dimension as a regular observation. Note that the definition of h_t^l slightly differs

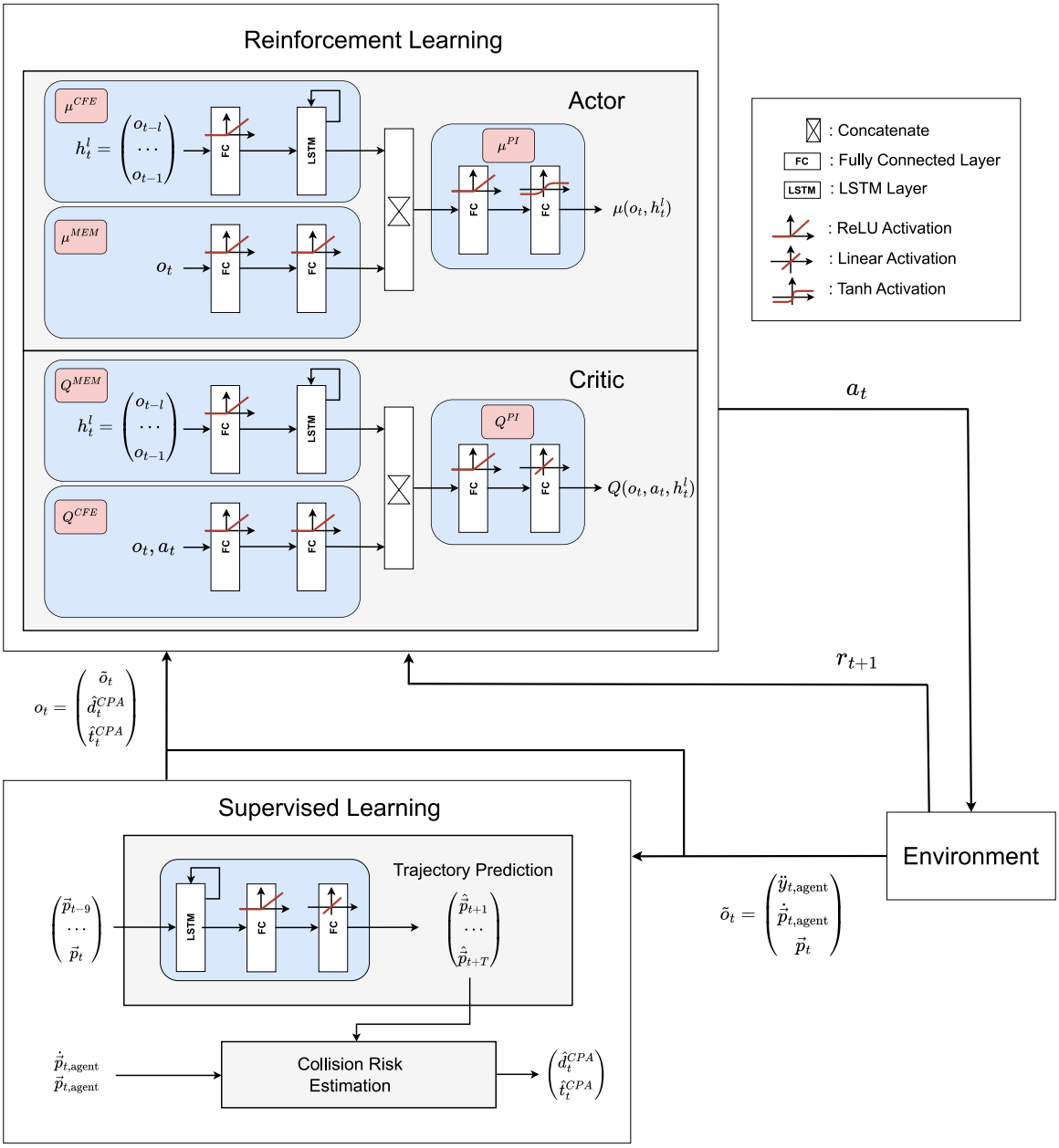


Fig. 1. The proposed two-level architecture for DOA tasks.

from [93] since we do not include past actions in the history. Furthermore, we set $l = 10$ throughout the paper, analogous to the history length of the trajectory prediction from Section 5.1. The algorithm disassembles both actor and critic into different sub-components. Precisely, there is a memory extraction (MEM) part in the function approximators, Q^{MEM} and μ^{MEM}, respectively, that processes the history. In parallel, the current feature extraction (CFE) components Q^{CFE} and μ^{CFE} process the observation of the current step o_t. Finally, the output of both MEM and CFE are concatenated and fed into the perception integration (PI) components Q^{PI} and μ^{PI}. These aggregate the extracted pieces of information and yield the final result. The network design of our LSTM-TD3 implementation is included in Fig. 1 and formalized as follows:

$$Q(o_t, a_t, h_t^l) = Q^{\text{PI}}\left\{Q^{\text{MEM}}(h_t^l) \bowtie Q^{\text{CFE}}(o_t, a_t)\right\},$$
$$\mu(o_t, h_t^l) = \mu^{\text{PI}}\left\{\mu^{\text{MEM}}(h_t^l) \bowtie \mu^{\text{CFE}}(o_t)\right\},$$

where $\bowtie$ is the concatenation operator. The optimization of the algorithm is identical to the TD3. Moreover, to ensure the robustness of our architecture with respect to the algorithm, we apply the SAC algorithm of [95,96], which uses a stochastic actor and automated tuning of the algorithm's temperature parameter. We construct the network design analogous to the LSTM-TD3 networks and refer to the approach in the following as LSTM-SAC.

5.4. Training environment

We aim to develop a challenging RL training environment for DOA where the agent has to anticipate several obstacle trajectories to avoid collisions. To solely focus on the collision avoidance task and ensure the agent does not leave the environmental area of interest, we define an environment where obstacles should be passed in a predefined fashion, so-called *passing rules*. Moreover, research on RL-based DOA

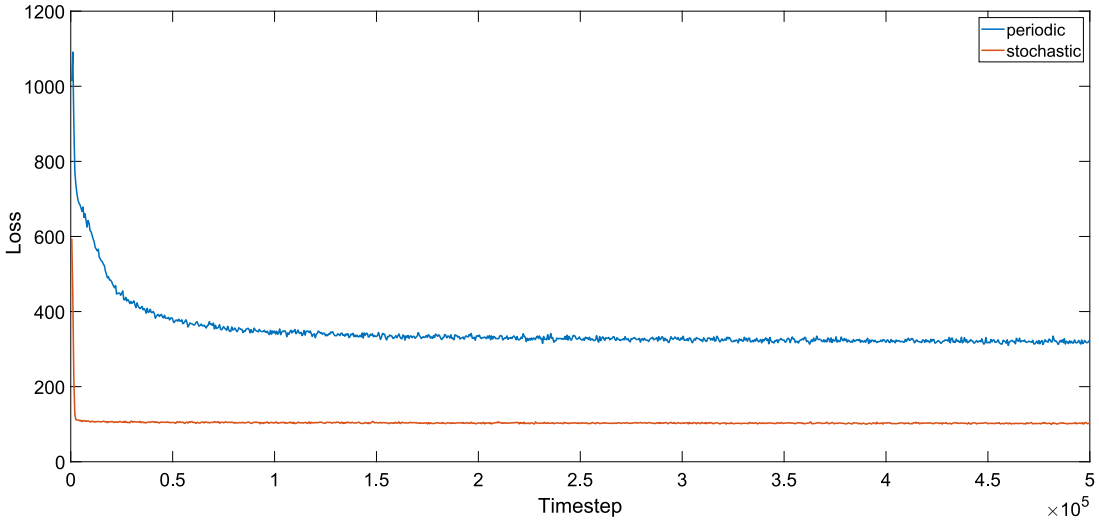


Fig. 2. Training of the trajectory prediction module for periodic and stochastic obstacle trajectories.

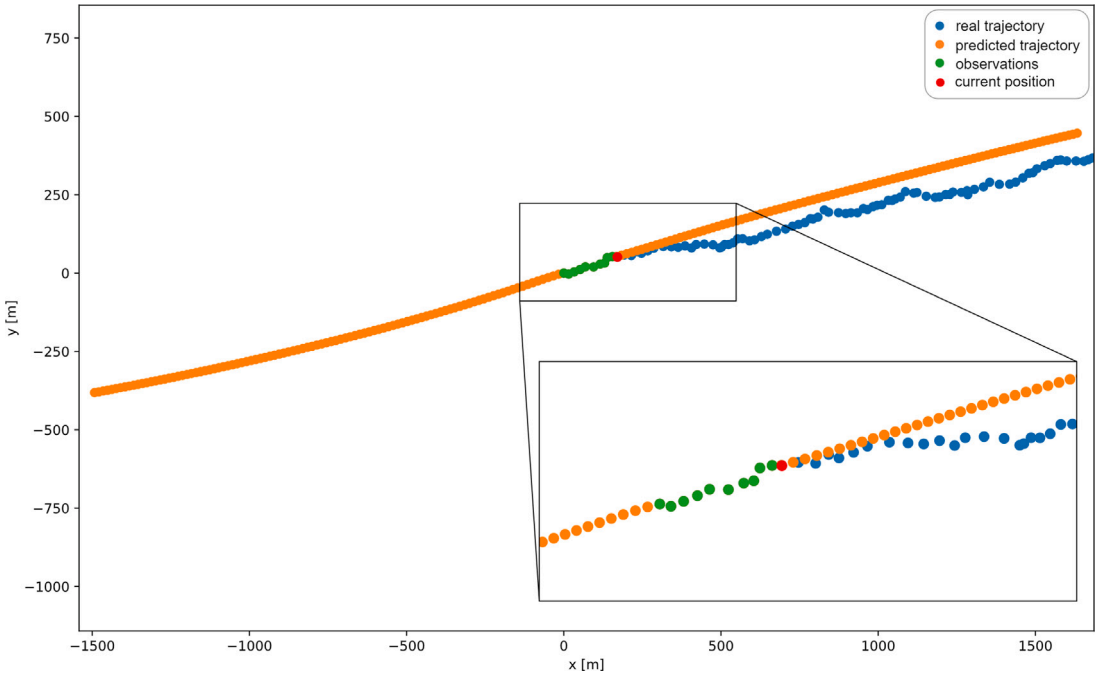


Fig. 3. Trajectory prediction for stochastic obstacle movement with last observations in green and ground truth data in blue.

indicates that the final performance increases when the CR in training is high [97], which we consider appropriately.

We consider a set of obstacles $\mathcal{M} = \{1, \dots, N_{\text{obstacle}}\}$, where $N_{\text{obstacle}} = 10$ is the total number of obstacles in the environment and the obstacle dynamics follow (1). We further define an agent with position $\vec{p}_{t,\text{agent}} = (x_{t,\text{agent}}, y_{t,\text{agent}})^\top$ and velocity $\dot{\vec{p}}_{t,\text{agent}} = (\dot{x}_{\text{agent}}, \dot{y}_{t,\text{agent}})^\top$ at time step t, where $\dot{x}_{\text{agent}}$ denotes a constant longitudinal velocity component. The RL agent controls the lateral velocity component via the lateral acceleration $\ddot{y}_{t,\text{agent}}$. Formally, the agent computes an action $a_t \in [-1, 1]$ each time step t that is mapped to the lateral acceleration:

$$\ddot{y}_{t,\text{agent}} = a_{y,\max} a_t,$$

where $a_{y,\max}$ defines the maximal lateral acceleration for the agent. The agent receives an observation vector each time step t, defined as:

$$o_t = \left(\frac{\ddot{y}_{t,\text{agent}}}{a_{y,\max}}, \frac{\dot{\vec{p}}_{t,\text{agent}}}{v_{\max}}, \frac{\vec{p}_{t,\text{agent}} - \vec{p}_{t,i}}{p_{\text{scale}}}, \frac{\hat{d}^{\text{CPA}}_{t,i}}{d^{\text{CPA}}_{\text{scale}}}, \frac{\hat{t}^{\text{CPA}}_{t,i}}{t^{\text{CPA}}_{\text{scale}}} \right)^\top, \tag{7}$$

where $v_{\max}$ is the maximum speed and $\vec{p}_{t,i}$, $\hat{d}^{\text{CPA}}_{t,i}$, and $\hat{t}^{\text{CPA}}_{t,i}$ denote the position, estimated d^{CPA}, and estimated t^{CPA} of the ith obstacle, respectively. Crucially, the estimates $\hat{d}^{\text{CPA}}_{t,i}$ and $\hat{t}^{\text{CPA}}_{t,i}$ are thereby computed using the supervised learning module. The constants p_{scale}, $d^{\text{CPA}}_{\text{scale}}$, and $t^{\text{CPA}}_{\text{scale}}$ are used for scaling. Since the computation of $\hat{d}^{\text{CPA}}_{t,i}$ and $\hat{t}^{\text{CPA}}_{t,i}$ is demanding, we update those values only every 10th simulation step. We emphasize that (7) slightly abuses notation and contains information about all obstacles $i \in \mathcal{M}$. Consequently, o_t is of dimension $3+4N_{\text{obstacle}}$.

The passing rule for each obstacle is indicated by the index within the observation vector. The first three observations are reserved for agent-related observations, the following $4N_{\text{obstacle}}/2$ observations are reserved for obstacles with passing rule 'right', and the last $4N_{\text{obstacle}}/2$ for obstacles with passing rule 'left'. Within these blocks of the same passing rule, obstacles are sorted according to ascending estimated $\hat{t}^{\text{CPA}}$. To validate the proposed approach, we additionally configure a baseline agent with a reduced observation space, neglecting $\hat{d}^{\text{CPA}}_{t,i}$ and

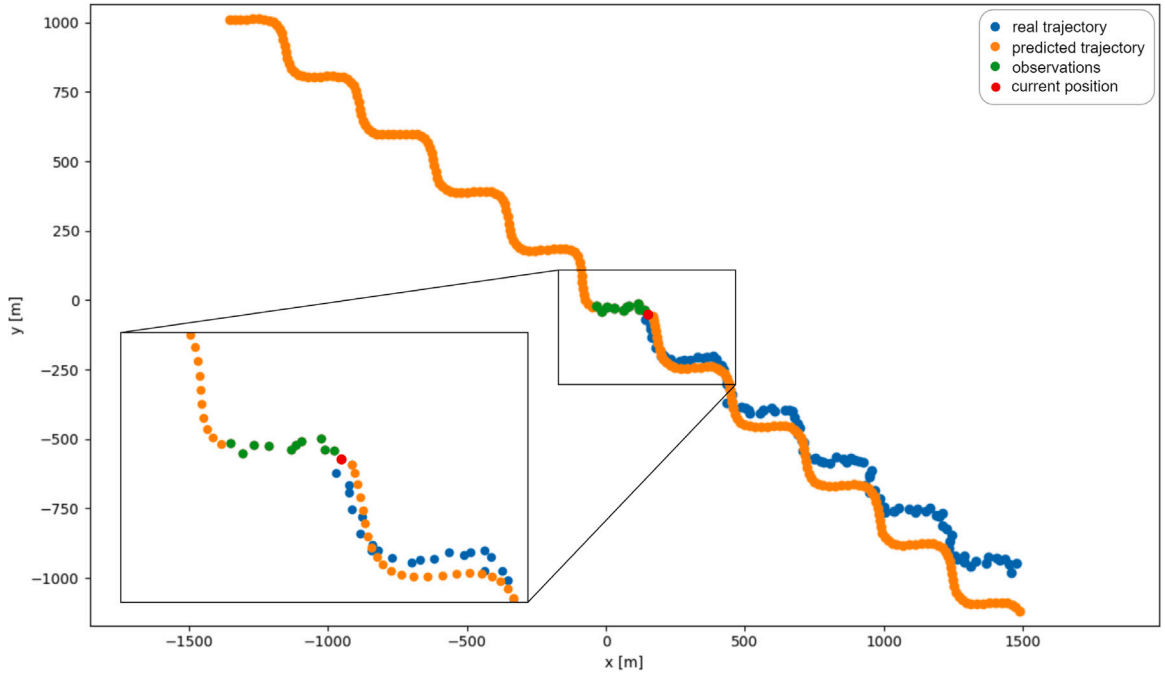


Fig. 4. Trajectory prediction for periodic obstacle movement with last observations in green and ground truth data in blue.

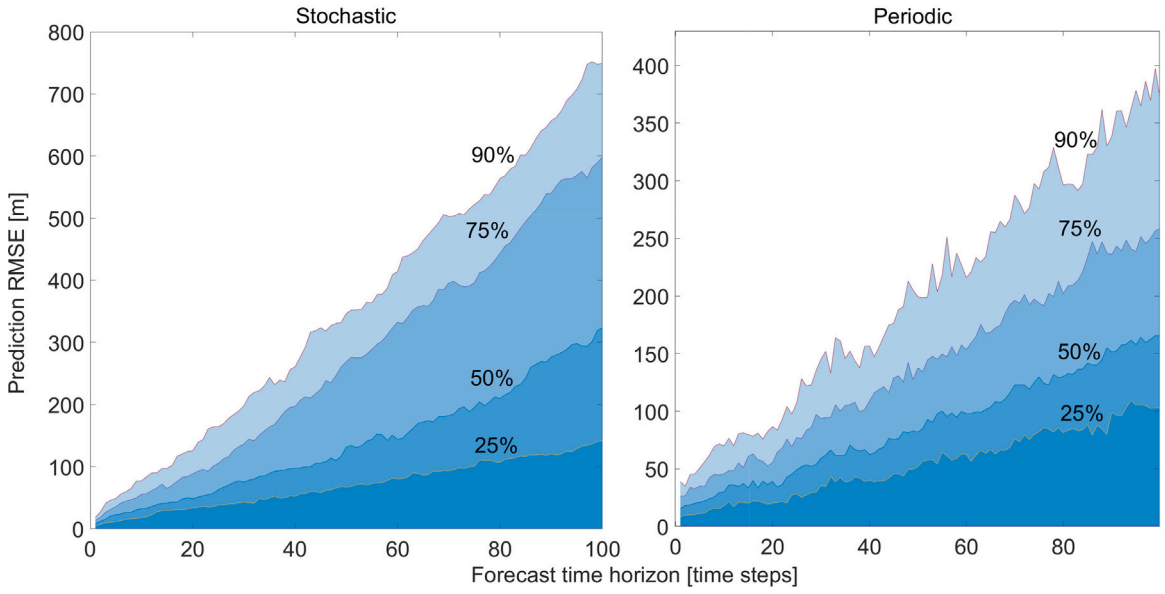


Fig. 5. RMSE between predicted trajectory and real trajectory with 25%-, 50%-, 75%-, and 90%-quantile.

$\hat{t}_{t,i}^{\mathrm{CPA}}$, and with sorting the obstacles according to ascending euclidean distance to the agent.

Following recent RL literature [98], we aim to keep the reward function as simple as possible. Therefore, we penalize the agent only when passing an obstacle on the wrong side:

$$r_{t+1} = \begin{cases} -1, & \text{passing an obstacle contrary to the respective passing rule,} \\ 0 & \text{else.} \end{cases} \tag{8}$$

Appendix B provides details about the initialization of the agent and the obstacles and other environment configurations. Fig. 7 exemplary depicts a time interval of a training episode, where blue and green colors refer to obstacles with different passing rules.

6. Experiments

6.1. Setup

We compare our proposed two-step architecture with a training procedure without incorporating the estimates of d^{CPA} and t^{CPA} in the observation vector. Crucially, to demonstrate the proposal's invariance to the selected DRL algorithm, we run the same experiments on two different actor-critic algorithms: the LSTM-TD3 of [93], which uses a deterministic actor, and the LSTM-SAC as described in Section 5.3, which uses a stochastic actor. All experiments are conducted in the same training environment, which is described in Section 5.4. Hyperparameters shared by both algorithms are set equal; see Table A.1. Fig. 8 displays the training results using a history length of $l = 4$,

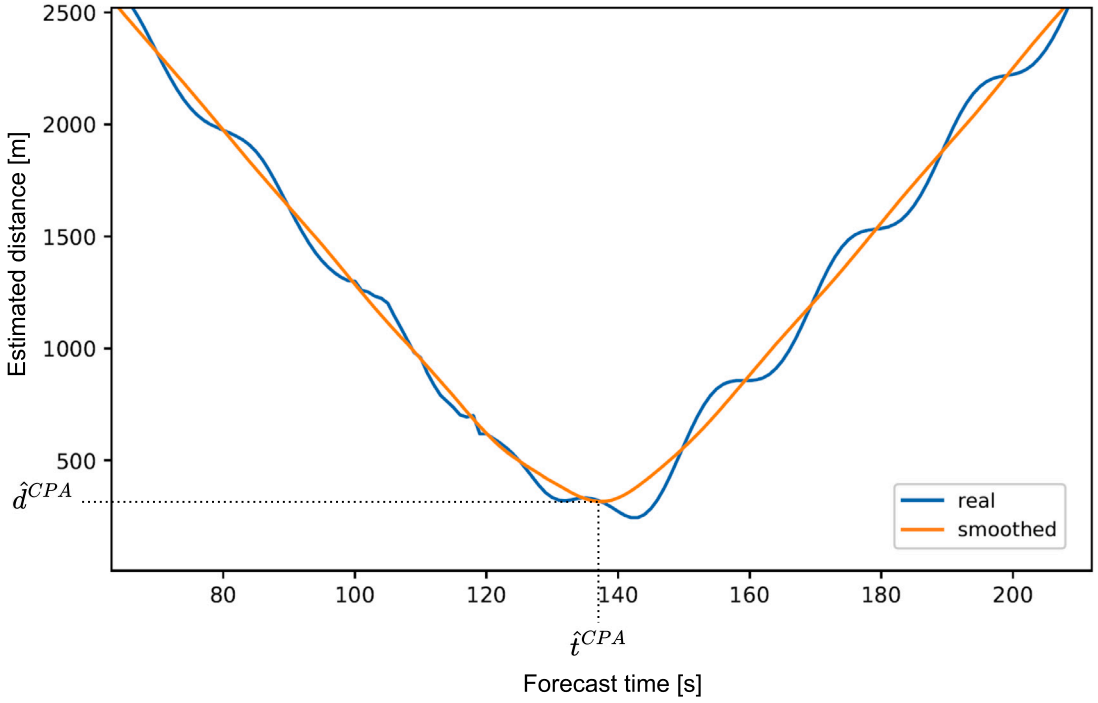


Fig. 6. Real and smoothed distance between a predicted obstacle and predicted agent trajectory for a periodic obstacle movement.

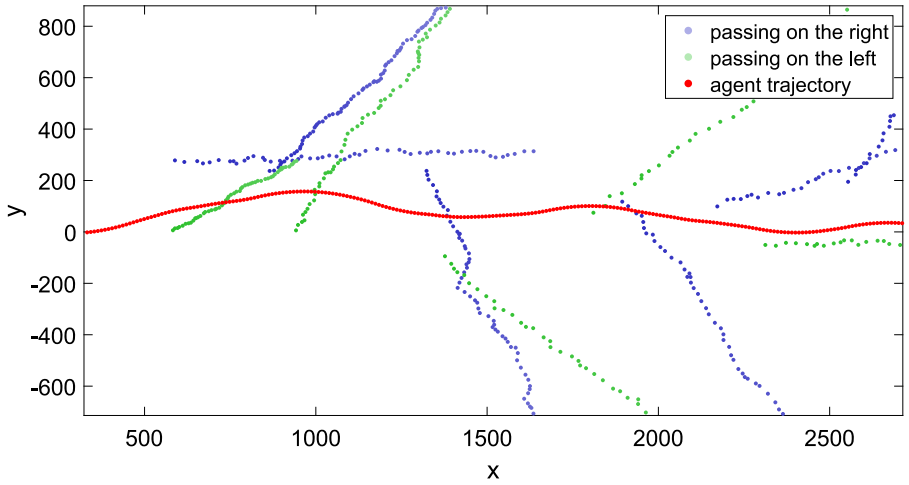


Fig. 7. Training environment with several obstacle trajectories (green and blue) and a possible collision-free agent trajectory (red).

where we mark agents trained with CR estimates from the supervised learning module with the prefix 'SL'. The results are averaged over ten independent runs of each experiment, and we include point-wise 95% confidence intervals for robustness. In addition, we include an ablation study over the history length, which is shown in Fig. 9.

6.2. Analysis

Following (8), the absolute value of the test return of an episode can be immediately interpreted as the number of collisions. Thus, according to Fig. 8, including the CR estimates based on supervised learning roughly halves the number of collisions. This performance boost is independent of the used algorithm, and the variance of the test returns is relatively small for $l = 4$. Analyzing Fig. 9, we see that the results for larger history lengths underline the finding from $l = 4$ that including the SL module is highly beneficial. In particular, for $l = 10$, the LSTM-TD3 without the SL component fails to learn anything, while the SL-LSTM-TD3 approach achieves much better performance, although at the expense of increased variance. We generally observe that the LSTM-TD3 has a higher return variance than the LSTM-SAC.

We visualize an exemplary episode for stochastic and periodic obstacle movements in Figs. 10 and 11, respectively, to give further intuition about how the trained agents perform in the obstacle avoidance environment. For illustration purposes, we show the trajectories of the LSTM-SAC with and without the SL module. The agent trajectories are depicted in red and yellow, and obstacle trajectories in blue and green, respectively, according to the imposed passing rules. The violations of the passing rules are marked with a black square. We observe that the agent equipped with the additional CR estimates (prefix 'SL') consistently selects better trajectories, avoiding violations of passing rules. In contrast, the baseline agent struggles to accurately predict the trajectories of some obstacles, leading to rule violations.

For instance, in Fig. 10, shortly after $x = 3000$ m, the SL-LSTM-SAC agent successfully navigates past an upcoming obstacle on the right,

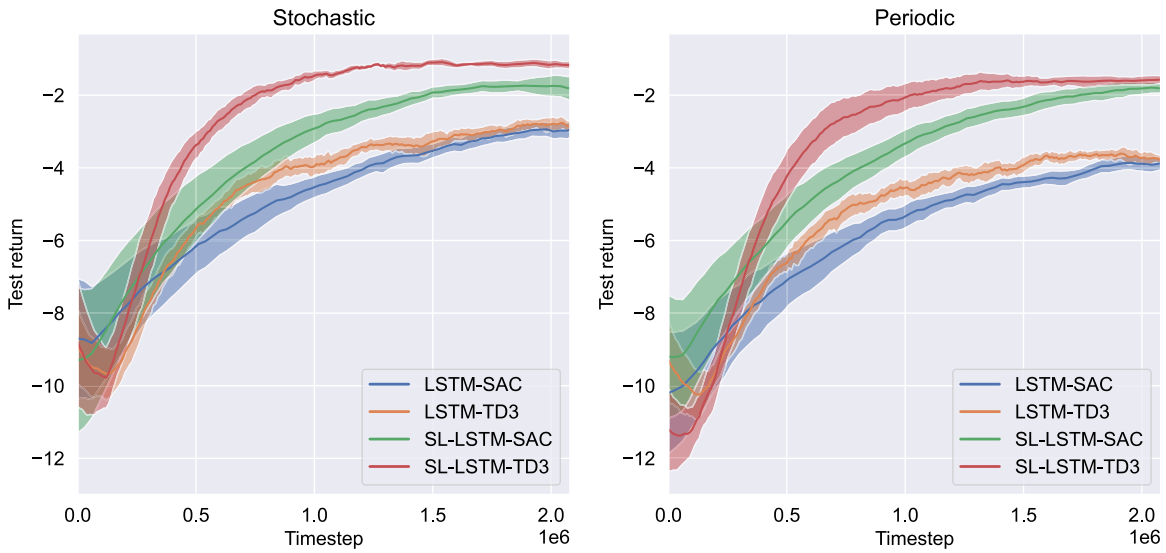


Fig. 8. Performance comparison of agents that use the estimates for d^{CPA} and t^{CPA} (prefix 'SL') with baseline agents without the SL subroutine for stochastic (left) and periodic (right) obstacle movement. The y-axis displays the test return during an episode and the history length is $l = 4$.

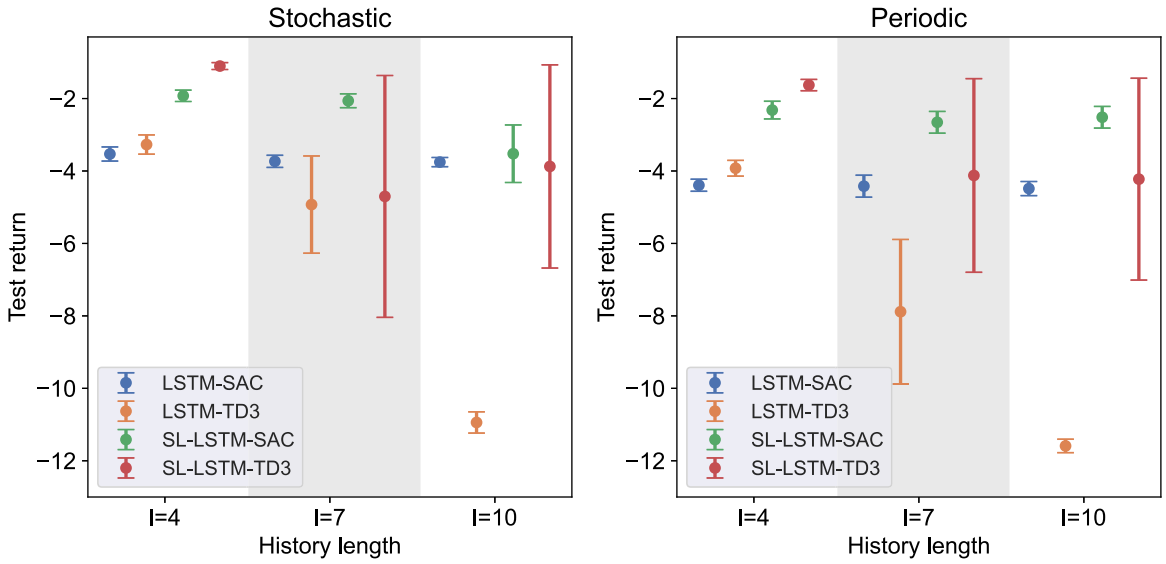


Fig. 9. Ablation study over the history lengths $l \in \{4, 7, 10\}$. The plots show the test return for the two obstacle movements after $1.5 \cdot 10^6$ training steps, including a 95% confidence interval based on 10 independent runs.

whereas the baseline agent's lateral position is too high, resulting in a rule violation. Later in the episode, after $x = 7000$ m, the SL-LSTM-SAC agent correctly passes an obstacle on the left and another on the right, while the baseline agent violates the rules by passing both obstacles on the right side. Similar findings are evident in Fig. 11. For example, at a longitudinal position slightly beyond $x = 9000$ m, the baseline agent incorrectly crosses an obstacle on the right side, violating the passing rules, while the SL-LSTM-SAC agent correctly passes on the left-hand side. These examples illustrate the superiority of the SL-LSTM-SAC agent in adhering to passing rules and choosing adequate trajectories.

7. Generalization study: Maritime traffic

7.1. Trajectory data

We demonstrate the generalizability of our two-step proposal to DOA by investigating the maritime traffic domain. In particular, we design an environment where the agent controls a vessel and needs to resolve head-on situations with high CR. Head-on scenarios are maritime encounter situations where two vessels approach each other on reciprocal or nearly reciprocal courses, posing a high risk of collision if neither vessel takes evasive action [26]. The obstacle vessel follows a real-world trajectory obtained from the Automatic Identification System (AIS, [99]), whose data we thankfully received from the European Maritime Safety Agency. The AIS is a tracking system used to identify and localize vessels, containing information such as positions, speeds, headings, timestamps, destinations, and ship types, among other details. Passenger ships and cargo ships above certain sizes must carry an AIS transponder and emit data to the system [100].

Acknowledging that AIS records may contain redundant or incomplete information [101], we use the package of [102] to employ filtering techniques to eliminate duplicate records and entries with missing information. As a use case, we filter vessel trajectories from the year 2021 in a specific area close to the city of Ebeltoft in Denmark. In total, 137 different trajectories have been extracted, and subsequent waypoints in a trajectory are interpolated based on cubic splines to

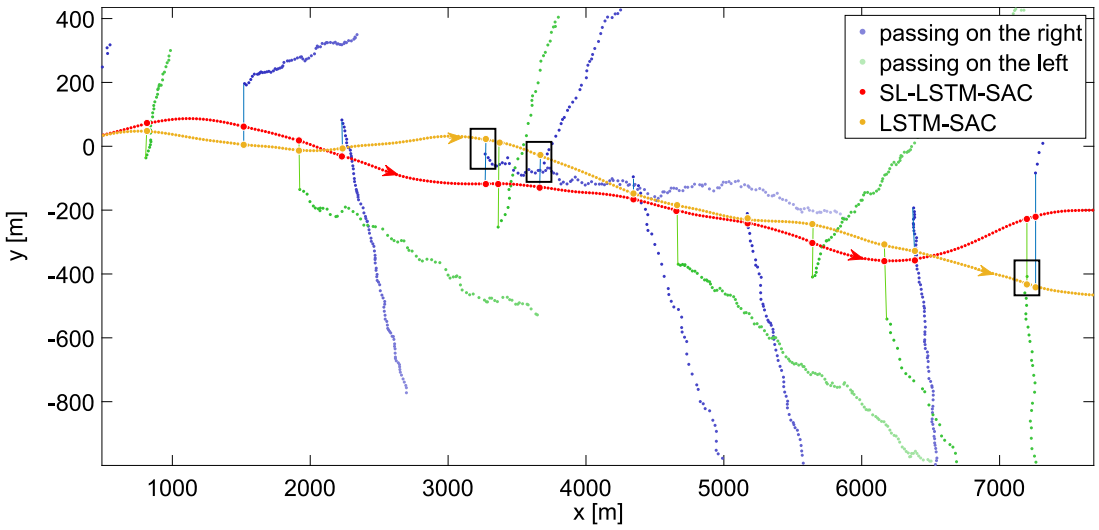


Fig. 10. Agent trajectories in the obstacle avoidance environment with stochastic obstacle movement, black squares indicating a violation of passing rules.

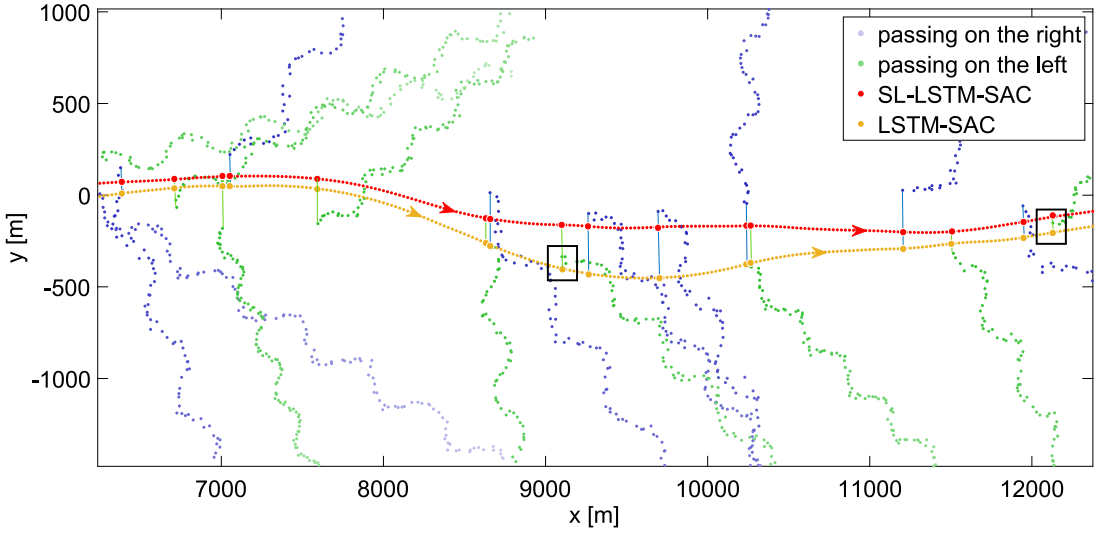


Fig. 11. Agent trajectories in the obstacle avoidance environment with periodic obstacle movement, black squares indicating a violation of passing rules.

be $\Delta t = 5$ s apart. As visualized in Fig. 12, the area features vessel trajectories characterized by highly non-linear behavior. Vessels need to navigate a curve to reach the open sea from the Ebeltoft harbor, making it a suitable candidate to demonstrate our two-step approach to DOA under non-linear obstacle movements.

7.2. Supervised learning

The supervised learning module adheres to the trajectory prediction and CR estimation framework outlined in Section 5. However, accounting for the increased complexity of the real-world trajectories, we empirically determined that two adjustments became necessary to maintain a high prediction quality. First, we increase the network size. In Fig. 1, the trajectory prediction network comprises an LSTM layer with 64 hidden units, followed by two linear layers with 64 neurons each, using ReLU and linear activation functions, respectively. For this case study, we expand the LSTM layer to 256 hidden units and employ three linear layers with 256 neurons each. Additionally, the first two linear layers now use leaky ReLU instead of a conventional ReLU activation function [103].

Second, the trajectory prediction network described in Section 5.1 fed the LSTM unit with the first differences of the last 10 obstacle positions $(\vec{p}_{t-9}, \dots, \vec{p}_t)^\top$. The output of the LSTM was then passed through the linear layers to estimate the next obstacle position in $\mathbb{R}^2$. In contrast, in this section, we recursively process the relative differences of the last 75 obstacle positions $(\vec{p}_{t-74}, \dots, \vec{p}_t)^\top$ using the LSTM. Considering a time step consists of 5 s, 75 observations equate to information about the past 6 min 25 s, a realistic time horizon for vessel traffic.

In addition, we concatenate the LSTM's output with the absolute position $\vec{p}_t$. This process can be understood as a form of 'anchoring'. As illustrated in Fig. 12, absolute position information is necessary to determine whether a curve should be forecasted, despite the general stability of predicting relative positions. Thus, we opt for this hybrid approach to leverage the strengths of both methods. Additionally, to save computation time due to the increased network size, we directly forecast the next 10 relative obstacle positions. Consequently, the new trajectory prediction module outputs in $\mathbb{R}^{10\cdot 2} = \mathbb{R}^{20}$.

During training, we randomly sample segments from the trajectories and add Gaussian noise to the positions to reduce overfitting. Visualizations of the resulting trajectory estimates after training are shown in Fig. 13. Overall, the predictions are accurate and capture the non-linear behavior well. However, the necessity of the anchoring mechanism is

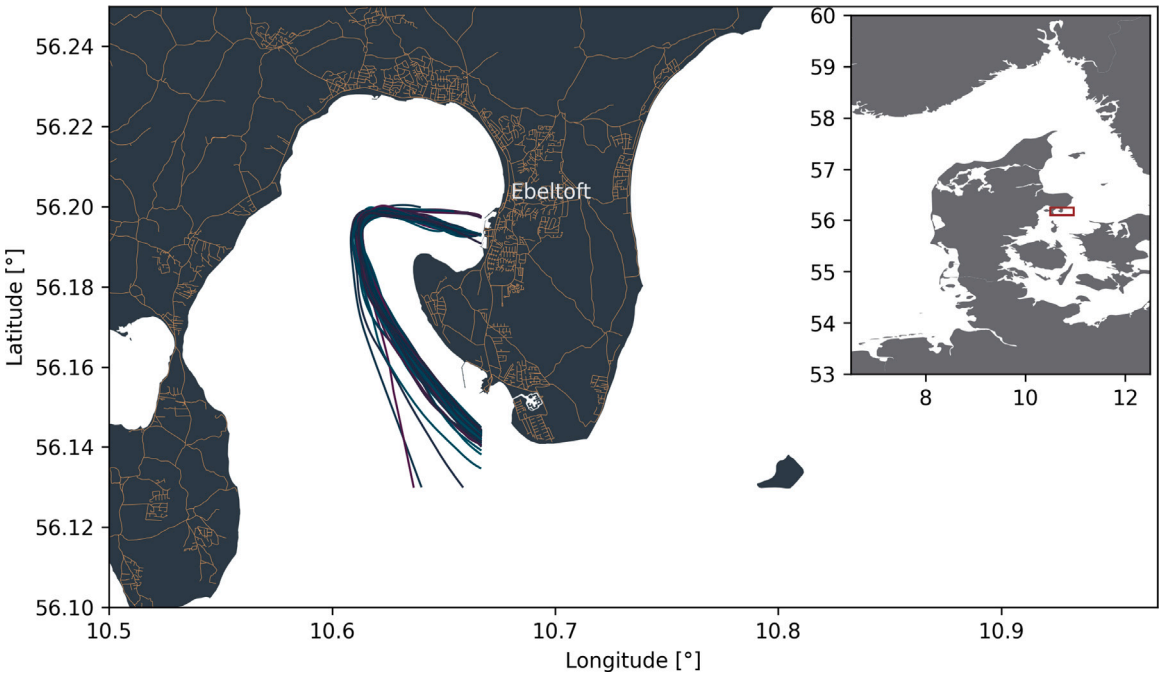


Fig. 12. The real-world AIS trajectories in the maritime area close to Ebeltoft, Denmark.

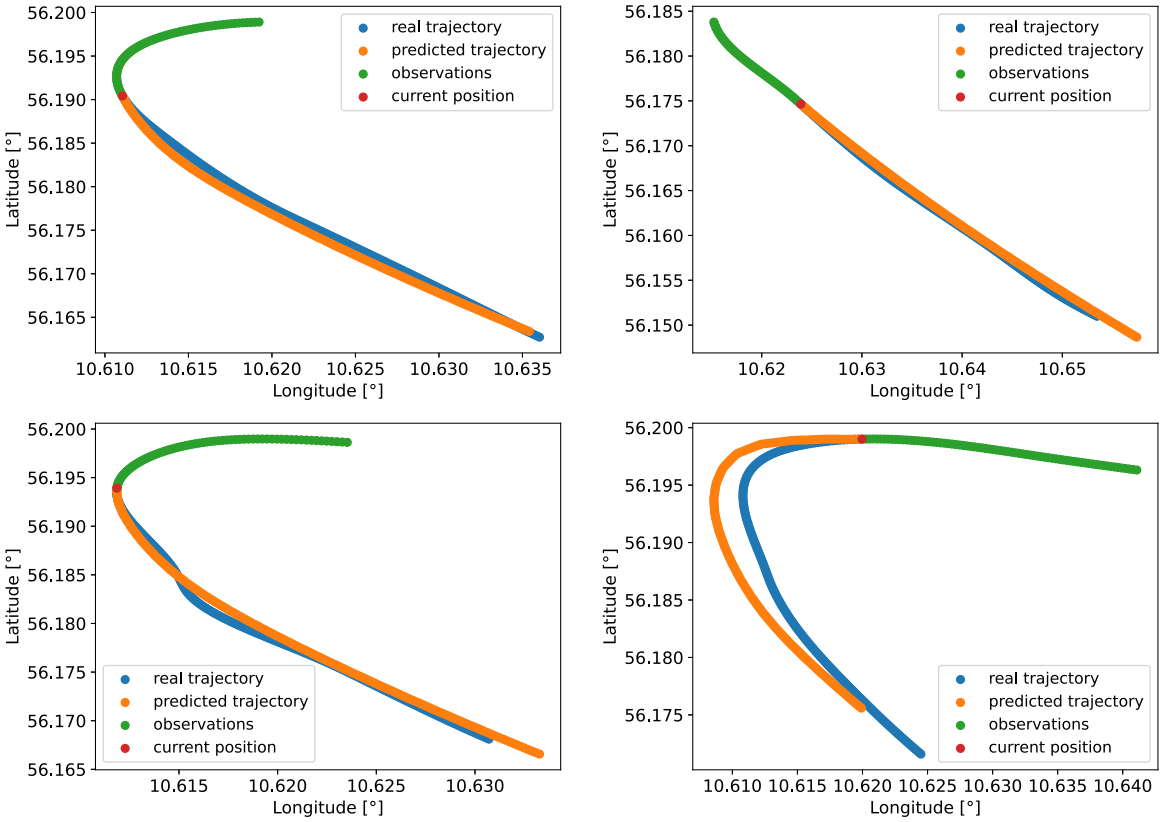


Fig. 13. Examples of the trajectory estimates using the supervised learning module.

evident in the lower right plot of Fig. 13. The observations in green, used to make predictions in orange, show no signs of a curve, making it difficult for the neural network to anticipate when the curve will appear. Without the anchoring, forecasting the curve would be almost impossible.

Similar to Fig. 5, we include a quantitative evaluation based on the RMSE over time in Fig. 14. In particular, we randomly sample 100 trajectory segments, forecast the next 200 time steps, and compare the predictions with the true trajectory points. Note that 200 time steps equate approximately 16.67 min. Following Fig. 14, we have a median RMSE of approximately 60 m over the time horizon.

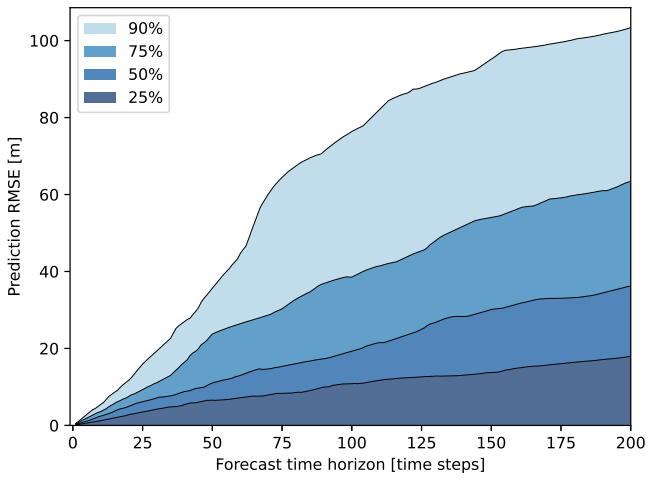


Fig. 14. RMSE between predicted and actual trajectory for different quantiles.

7.3. Reinforcement learning environment

The maritime RL environment requires the agent to resolve head-on situations with an obstacle vessel, as visualized in Fig. 15. At the beginning of each episode, we randomly sample an obstacle trajectory, corresponding to Obstacle 1 in the figure. We then determine, with equal probability, whether Obstacle 1 is initialized before or after the curve, depicted in the left or right plot in Fig. 15, respectively. In both scenarios, the agent is penalized if it avoids Obstacle 1 via a portside (left) turn since such a maneuver does not comply with maritime traffic rules for head-on situations [26]. Consequently, the agent can develop the following strategy: If Obstacle 1 is set to perform a curve, the agent can maintain its course; otherwise, it should execute an avoidance maneuver to the starboard (right) side.

In addition, we randomly generate three other obstacles in each episode, which move linearly and are included to increase the complexity of the problem. The agent has a speed of approximately 8 m/s and is initialized with a time-to-collision of 4 min to the head-on vessel. Furthermore, we add Gaussian noise to the initial positions of the agent and obstacle vessels to ensure the diversity of the episodes. As in Section 6, we compare RL agents with the CR estimates from the SL module with baseline agents without those estimates.

The observation vector of an agent at time t is defined as follows:

$$o_t = \left(\frac{U_{t,\text{agent}}}{U_{\text{scale}}}, \frac{y^e_{t,\text{agent}}}{y^e_{\text{scale}}}, \frac{\vec{d}_{t,i} - d_{\text{coll}}}{d_{\text{scale}}}, \frac{\vec{\alpha}_{t,i}}{\pi}, \frac{\vec{\psi}_{t,i} - \psi_{t,\text{agent}}}{\pi}, \frac{\vec{U}_{t,i}}{U_{\text{scale}}}, \frac{\hat{d}^{\text{CPA}}_{t,i} - d_{\text{coll}}}{d^{\text{CPA}}_{\text{norm}}}, \frac{\hat{t}^{\text{CPA}}_{t,i}}{t^{\text{CPA}}_{\text{norm}}} \right)^{\top}, \quad (9)$$

where $U_{t,\text{agent}}$ and $\psi_{t,\text{agent}}$ are the speed and heading angle of the agent. The cross-track error of the agent is denoted $y^e_{t,\text{agent}}$ and constitutes the lateral offset to the linear path from the agent's starting position during an episode. This metric informs the agent whether it performs a starboard or portside avoidance maneuver. Furthermore, $\vec{d}_{t,i}$ is the Euclidean distance between the agent and obstacle i, $\vec{\psi}_{t,i}$ and $\vec{U}_{t,i}$ denote the heading angle and speed of obstacle i, and $\vec{\alpha}_{t,i}$ is the relative bearing of obstacle i from the perspective of the agent. For definitions of these metrics, we refer to the textbook of [8].

As in Section 5.4, $\hat{d}^{\text{CPA}}_{t,i}$ and $\hat{t}^{\text{CPA}}_{t,i}$ denote the estimates of the d^{CPA} and t^{CPA} with obstacle i from the SL module. The normalizing constants are set to $U_{\text{scale}} = 5$ m/s, $y^e_{\text{scale}} = 200$ m, $d_{\text{coll}} = 300$ m, $d_{\text{scale}} = 3000$ m, $d^{\text{CPA}}_{\text{norm}} = 100$ m, and $t^{\text{CPA}}_{\text{norm}} = 100$ s. Considering that there are four obstacles in the environment, the length of o_t is $2+4\cdot 6 = 26$. We sort o_t with respect to the ascending estimated d^{CPA}. For the baselines without the CR estimates, we sort o_t with respect to the ascending Euclidean distance. We emphasize that only the CR estimates for Obstacle 1 in Fig. 15 are computed via the SL module. Since the other obstacles move linearly, using the analytic formulas of [30] suffices.

The action of an agent at time t, a_t, modifies the agent's heading:

$$\psi_{t+1,\text{agent}} = \psi_{t,\text{agent}} + a_t \cdot a_{\text{scale}},$$

where the value $a_{\text{scale}} = 2.4°$ is carefully chosen to ensure the agent's maneuverability is limited enough to create a difficult situation while ensuring the problem remains solvable. Furthermore, we allow the agent to select an action only every four time steps to ensure the maneuverability is not unrealistically high.

Lastly, the reward function is computed via:

$$r_{t+1} = -\mathbf{1}\{y^e_t \geq y^e_{\text{scale}}\} - \sum_{i=1}^{4} \mathbf{1}\{d_{i,t} \leq d_{\text{coll}}\},$$

with the indicator function $\mathbf{1}\{x\}$ being one if x is true and zero otherwise. The first term is the penalty for performing a portside maneuver, while the sum term relates to collision avoidance. An episode ends when 60 time steps, equating to 5 min, have passed.

7.4. Results

7.4.1. Setup

We run the LSTM-TD3 and LSTM-SAC algorithms for both configurations, with and without the estimates from the SL module. The hyperparameters are identical to those used in Section 6 and are listed in Table A.1. Additionally, we set the history length of the RL algorithms to $l = 4$ as it showed the best performance in Fig. 9. The return development during training is displayed in Fig. 16, where we average the returns of 15 independent runs and include 95% point-wise confidence intervals.

Furthermore, we investigate the robustness of the final policies of the different agents in the presence of positional noise. Specifically, at each time step, we disturb the sensed north and east positions of each involved vessel with independent realizations of a univariate Gaussian variable with zero mean and standard deviations $\sigma \in \{0, 5, 10, 20\}$ meters to reflect real-world sensor noise [9]. The state features in (9) were subsequently computed using the noisy positional observations. For each algorithm, noise level, and scenario in the environment, we run 100 randomly initialized test episodes and measure whether a collision occurred. The collision rates are presented in Fig. 17, including a confidence interval based on the 100 observations.

7.4.2. Analysis

Similar to the findings in Section 6, Fig. 16 demonstrates that the configurations with the SL prefix significantly outperform their conventional counterparts in terms of final return performance. Additionally, these configurations exhibit faster convergence and reduced variance in test returns. Moreover, Fig. 17 highlights the increased robustness when leveraging the SL estimates. Both approaches, with and without SL, generally learn the correct strategies of maintaining the course in Scenario 1 and turning starboard in Scenario 2. However, even in the case $\sigma = 0$ m, which corresponds to the noise-free training scenarios, the DRL agents without the SL estimates more frequently misinterpret Scenario 1 and produce a collision. Moreover, the SL-based approaches show remarkable resilience to increased positional noise. In contrast, the performance of the conventional approaches degrades significantly, resulting in much higher collision rates, especially in Scenario 2.

8. Discussion and outlook

The results of the investigations in Sections 6 and 7 display the performance potential of our proposed two-step approach to DOA. In particular, DRL is especially promising for real-time DOA tasks due to its adaptability and capability of handling a wide range of environments and obstacle configurations without explicit programming for each scenario. In addition, the fast inference time of neural networks makes our approach highly practical for real-time DOA situations.

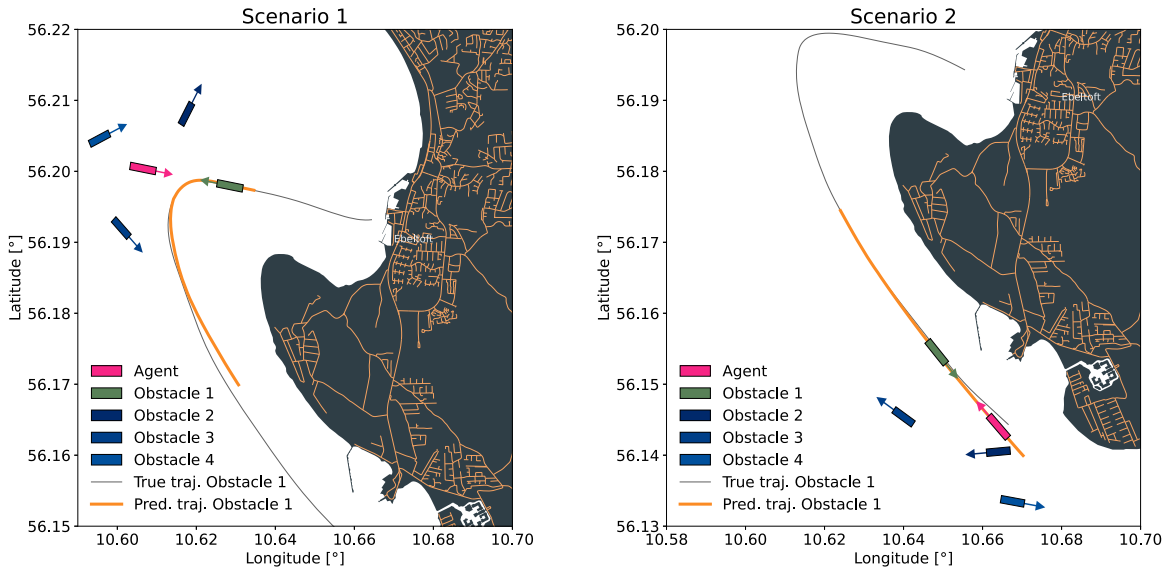


Fig. 15. The two possible scenarios in the maritime traffic environment. The gray solid line is the true trajectory of Obstacle 1, while the predicted trajectory using the SL module is shown in orange. Obstacle 1 performs a portside curve in the left plot while it travels almost straight in the right plot.

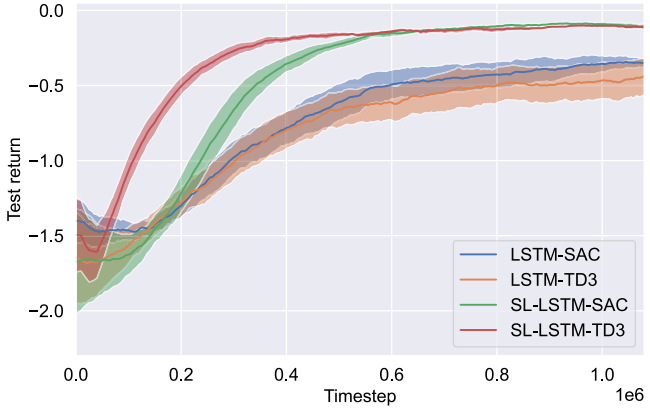


Fig. 16. Performance comparison of agents that use the estimates for d^{CPA} and t^{CPA} (prefix 'SL') with baseline agents without the SL subroutine for the maritime traffic environment. The y-axis displays the test return during an episode.

However, we acknowledge the limitations of our work that can be addressed in future research. In particular, we focus on non-cooperative obstacle behavior, where the obstacles themselves do not perform active collision avoidance maneuvers. Our two-step architecture could handle such scenarios if historical data on obstacle behavior in specific encounter situations is available, and future research may validate the proposal in such cases. Furthermore, the static nature of historical data does not account for potential changes in obstacle behavior over time due to evolving dynamics or new traffic patterns. Implementing a mechanism to periodically update the supervised learning model with new data and trends would ensure that the system remains effective and relevant in changing environments. This continuous learning approach would allow the model to adapt to new obstacle behaviors as they emerge, maintaining high levels of safety and efficiency in obstacle avoidance.

In addition, we focus on non-communicative situations where the obstacles do not explicitly inform the agent about their intentions, necessitating the prediction of their future trajectories. If such information channels were available, as assumed in [104], the decision-making of the DRL agent can be further improved. Nevertheless, a non-communicative system, like ours, is still essential to handle cases when the communication channel fails [105]. Lastly, our proposal provides point estimates of the d^{CPA} and t^{CPA} risk metrics. While this procedure strongly improves the performance of the respective DRL agent, we acknowledge that the CR estimates critically depend on the uncertainty connected to the predicted obstacle trajectories. To address this, it might be beneficial to derive confidence bounds for the CR estimates by considering sampling-based approaches [106] or imposing distributional assumptions [107] for the trajectory prediction. This addition might allow for better-informed decision-making by quantifying the reliability of the risk assessments and accounting for potential variations in obstacle behavior.

9. Conclusion

Future sustainable transportation systems can benefit from autonomous vehicles in terms of safety and energy efficiency, particularly in non-lane-based domains such as air or maritime traffic. Such real-world autonomous systems necessitate robust local path planning capabilities, including the critical task of avoiding collisions with dynamic obstacles. This work proposes a two-step architecture that leverages recent advances from supervised learning and reinforcement learning. The first step captures non-linear movements of obstacles and estimates CR metrics in a supervised fashion, while the second step uses these estimates to enrich the observation space of an RL agent.

Our approach has demonstrated a significant enhancement in the collision avoidance capabilities of the respective autonomous agent, thereby constituting a valuable contribution to the realm of autonomous vehicles. Importantly, our learning architecture exhibits versatility by not being confined to a specific set of obstacle dynamics. We validated our approach in a generic environment, independent of a particular traffic domain, and a maritime traffic environment using real-world vessel trajectory data. In our upcoming research, we aim to validate this architecture using a wider range of real-world obstacle behavior data and implement it across different traffic domains. Moreover, we

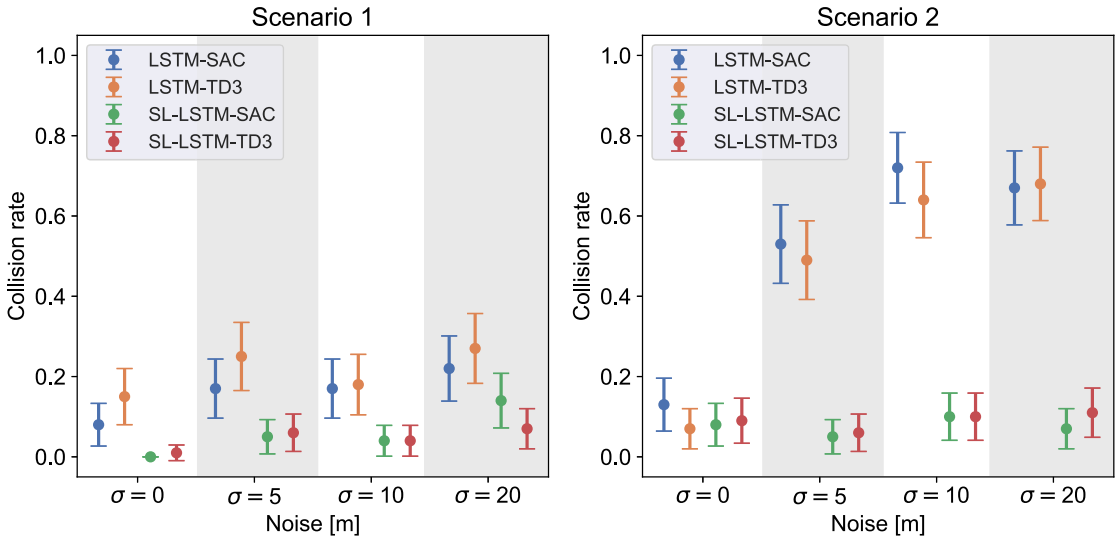


Fig. 17. Robustness analysis of the agents with respect to positional noise.

will continue to improve the proposed methodology via the routes outlined in Section 8, including the consideration of cooperative obstacle behavior and uncertainty quantification of the CR estimates.

CRediT authorship contribution statement

Fabian Hart: Writing – original draft, Software, Methodology, Conceptualization. **Martin Waltz:** Writing – original draft, Software, Methodology, Conceptualization. **Ostap Okhrin:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

Acknowledgments

This work was partially funded by BAW - Bundesanstalt für Wasserbau (Mikrosimulation des Schiffsverkehrs auf dem Niederrhein, award number 181.7.03_0004_02_2023). The authors are grateful to the Center for Information Services and High Performance Computing at TU Dresden for providing its facilities for high throughput calculations. Furthermore, the authors thank the European Maritime Safety Agency for providing the AIS data used for the generalization study of maritime traffic.

Appendix A. RL algorithm hyperparameters

See Table A.1.

Table A.1
List of hyperparameters of the RL algorithms.

Hyperparameter	Value
Discount factor	0.99
Batch size	32
Loss function	Mean squared error
Replay buffer size	10^6
Learning rate actor	0.0001
Learning rate critic	0.0001
Optimizer	Adam
Number of units per hidden layer	128
Target noise[a]	0.2
Target noise clip[a]	0.5
Policy update delay[a]	2
Initial temperature[b]	0.2
Learning rate temperature[b]	0.0001

[a] Solely used by the LSTM-TD3.
[b] Refers to LSTM-SAC.

Appendix B. Environment details

At the beginning of a training episode, we initialize the agent's dynamics to zero, except the longitudinal speed $\dot{x}_{\text{agent}}$, that is sampled uniformly at random from the interval $[1\ \text{m/s}, v_{\max}]$. The Euler and ballistic methods are used to update the agent's lateral speed and the positions for the agent and obstacles at time step $t+1$ [108]. Exemplary for the agent, we have:

$$\dot{y}_{t+1,\text{agent}} = \dot{y}_{t,\text{agent}} + \ddot{y}_{t+1,\text{agent}}\,\Delta t, \tag{B.1}$$

$$\vec{p}_{t+1,\text{agent}} = \vec{p}_{t,\text{agent}} + \frac{\dot{\vec{p}}_{t,\text{agent}} + \dot{\vec{p}}_{t+1,\text{agent}}}{2}\Delta t, \tag{B.2}$$

with $\Delta t = 5$ s corresponding to the simulation step size. One episode is defined as $N_{\text{steps}} = 500$ steps of updating.

For each obstacle, we define a passing rule. The set of obstacles that should only be passed, from the perspective of the agent, on the right side in the lateral direction is denoted as $\mathcal{M}_{\text{right}} = \{1, \dots, N_{\text{obstacle}}/2\}$. Consequently, the remaining obstacles should be passed left and are denoted $\mathcal{M}_{\text{left}} = \{N_{\text{obstacle}}/2 + 1, \dots, N_{\text{obstacle}}\}$.

Next, a procedure to update the trajectories of obstacles needs to be defined so that the agent has to face threatening obstacles constantly. Therefore, we replace obstacles that have already been passed. To specify this condition, we introduced the helper variable $TTC_{t,i}$ as the agent's time-to-collision with an obstacle $i \in \mathcal{M}$ in the longitudinal direction at time step t, where the time-to-collision is computed based on the obstacle's linear trajectory part (2). Negative values for $TTC_{t,i}$ relate to obstacles that have already passed the agent in the longitudinal direction. If for two obstacles $k, l \in \mathcal{M}_{\text{right}}$ holds: $TTC_{t,k} < 0$, $TTC_{t,l} < 0$, and $TTC_{t,k} < TTC_{t,l}$, we replace obstacle k as shown in

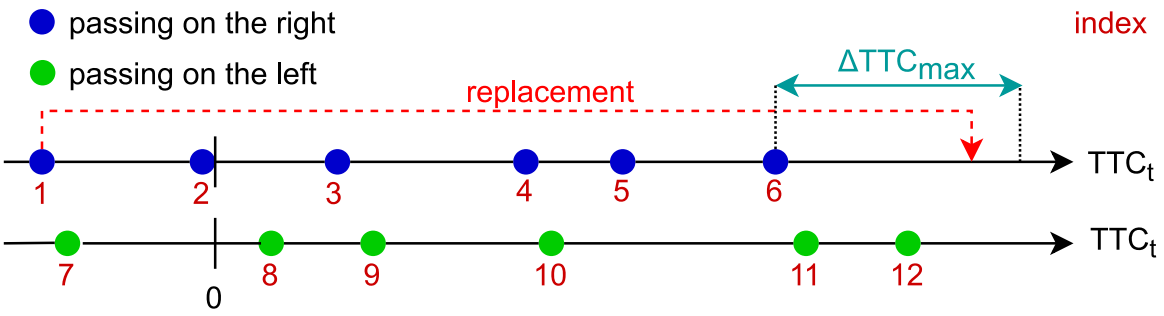


Fig. B.1. Replacement of an obstacle (obstacle 1) since two obstacles with the same passing rule already passed the agent (negative time-to-collision). The obstacle's new $TTC_{t,1}$ is set uniformly at random in the time interval colored turquoise with the length of $\Delta TTC_{\max}$.

Table B.1
Environment parameters.

Parameter	Description	Value
N_{obstacle}	number of obstacles	10
$a_{y,\max}$	agent's maximum lateral acceleration	0.01 m/s^2
$v_{\max}$	agent's maximum speed	5 m/s
Δt	simulation step size	5 s
N_{steps}	number of episode steps	500
p_{scale}	scaling factor for observation	3000 m
$d^{\text{CPA}}_{\text{scale}}$	scaling factor for observation	400 m
$t^{\text{CPA}}_{\text{scale}}$	scaling factor for observation	300 s
$\Delta TTC_{\max}$	maximal temporal distance for replacing an obstacle	300 s
ϕ_{stoch}	AR(1) process parameter	0.9
ϕ_{traj}	AR(1) process parameter	0.99
σ^2_{stoch}	normal distribution variance	225 m^2
σ^2_{traj}	normal distribution variance	800 m^2
σ^2_{sin}	normal distribution variance	16 m^2
β_{stoch}	smoothing factor	0.2
β_{traj}	smoothing factor	0.03
$\mu_{\Delta y}$	normal distribution mean	100 m
$\sigma^2_{\Delta y}$	normal distribution variance	2500 m^2
$\Delta y_{\min}$	minimum bound for Δy	40 m
A_{sin}	periodic trajectory amplitude	15 s
T_{sin}	periodic trajectory period	20 s

Fig. B.1. Its new time-to-collision is randomly sampled from:

$$TTC_{t,k} \sim \mathcal{U}\left(\max_{j\in\mathcal{M}_{\text{right}}}(TTC_{t,j}), \max_{j\in\mathcal{M}_{\text{right}}}(TTC_{t,j}) + \Delta TTC_{\max}\right), \tag{B.3}$$

where $\Delta TTC_{\max}$ is the maximal temporal distance for the new placement of an obstacle. This parameter affects the number of obstacles being passed in a certain time interval and is, therefore, a crucial design element of this environment. The same replacement procedure is applied for obstacles with passing rule 'left.'

Having computed the new value for $TTC_{t,i}$ for a replaced obstacle $i \in \mathcal{M}$ at time step t, the new position and velocity need to be determined. First, we draw values for the constant velocity $\dot{\vec{p}}_i = (\dot{x}_i, \dot{y}_i)^\top$ of the linear trajectory part from uniform distributions:

$$\dot{x}_i \sim \mathcal{U}(-v_{x,\max}, v_{x,\max}), \tag{B.4}$$

$$\dot{y}_i \sim \mathcal{U}(-v_{y,\max}, v_{y,\max}). \tag{B.5}$$

Second, the new longitudinal position can be set as follows:

$$x_{t,i} = (\dot{x}_{t,\text{agent}} - \dot{x}_i)TTC_{t,i} + x_{t,\text{agent}}. \tag{B.6}$$

Third, having the lateral speed of the replaced obstacle set, we generate the new lateral position with the help of a predefined, stochastic trajectory $y_{t,\text{traj}}$, inspired by [109]. This lateral trajectory is computed at the beginning of an episode and is based on a smoothed AR(1) process, whose parameters reflect the kinematics of the agent. Fig. B.2 shows a replacement situation identical to Fig. B.1 and illustrates how this trajectory is used to define the new lateral position for a replaced obstacle. One can think of this process as an exemplary trajectory the agent has to follow to avoid collisions with obstacles. In the following, we define the smoothed AR(1) process and give a detailed explanation of the replacement of an obstacle based on that process.

The AR(1) process is defined as:

$$B_{t+1} = \phi_{\text{traj}} B_t + u_t, \quad \text{where} \quad u_t \sim \mathcal{N}(0, \sigma^2_{\text{traj}}), \tag{B.7}$$

with auto-regressive parameter ϕ_{traj} and variance σ^2_{traj}. The parameters have been designed to model a lateral trajectory the agent can approximately follow under acceleration and velocity constraints represented by $a_{y,\max}$ and $v_{\max}$. To reduce the noise, we exponentially smooth the AR(1) process:

$$y_{t,\text{traj}} = \begin{cases} B_0, & \text{for } t = 0, \\ \beta_{\text{traj}} B_t + (1-\beta_{\text{traj}}) y_{t-1,\text{traj}}, & \text{for } t > 0, \end{cases} \tag{B.8}$$

where β_{traj} defines the smoothing factor. Based on this trajectory and having already computed $TTC_{t,i}$, $x_{t,i}$, $\dot{x}_i$, and $\dot{y}_i$ via (B.3), (B.4), (B.5), and (B.6), one more step is needed to set the new lateral position $y_{t,i}$ for a replaced obstacle $i \in \mathcal{M}$ at time step t.

We define Δy_i as the absolute difference between an obstacle's lateral position $y_{t,i}$ and the defined trajectory $y_{t,\text{traj}}$ when agent and obstacle are at the same longitudinal position ($TTC_{t,i} = 0$):

$$\Delta y_i = |y_{t,i} - y_{t,\text{traj}}| \quad \text{for} \quad TTC_{t,i} = 0, \tag{B.9}$$

shown yellow in Fig. B.2. To force our agent to move approximately along the trajectory $y_{t,\text{traj}}$, the positional difference Δy_i should be small, thus being another crucial design parameter to adjust the complexity of the environment. Every time an obstacle i is replaced, the variable Δy_i is sampled from a normal distribution:

$$\Delta y_i \sim \mathcal{N}(\mu_{\Delta y}, \sigma^2_{\Delta y}), \tag{B.10}$$

and lower-bounded to $\Delta y_{\min}$:

$$\Delta y_i = \max(\Delta y_{\min}, \Delta y_i). \tag{B.11}$$

By changing the parameters $\mu_{\Delta y}$, $\sigma^2_{\Delta y}$, and $\Delta y_{\min}$, one can adjust how close the obstacles are coming to the trajectory $y_{t,\text{traj}}$ when obstacle and agent are at the same longitudinal position. Table B.1 contains the chosen values for those parameters. Finally, the lateral position for obstacles $i_{\text{R}} \in \mathcal{M}_{\text{right}}$ is computed via:

$$y_{t,i_{\text{R}}} = y_{t,\text{traj}} + \Delta y_{i_{\text{R}}} - \dot{y}_{i_{\text{R}}} TTC_{t,i_{\text{R}}}, \tag{B.12}$$

and for obstacles $i_{\text{L}} \in \mathcal{M}_{\text{left}}$ via:

$$y_{t,i_{\text{L}}} = y_{t,\text{traj}} - \Delta y_{i_{\text{L}}} - \dot{y}_{i_{\text{L}}} TTC_{t,i_{\text{L}}}. \tag{B.13}$$

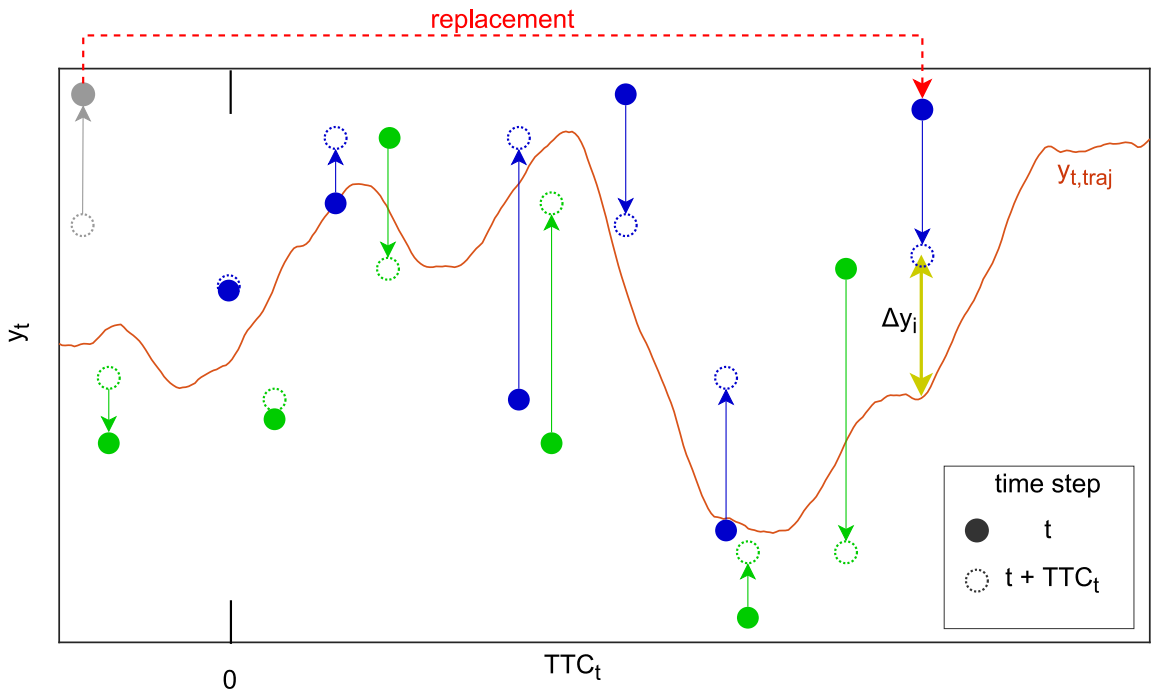


Fig. B.2. Replacement of an obstacle identical to the situation in Fig. B.1 but with additional information about lateral positions of obstacles.

Fig. B.2 shows the final lateral position and time-to-collision for a replaced obstacle as a filled circle. The linear trajectory part is represented as a blue or green arrow. Having defined the linear trajectory part of an obstacle, we can now add the non-linear trajectory part based on (1).

References

[1] S. Feng, H. Sun, X. Yan, H. Zhu, Z. Zou, S. Shen, H.X. Liu, Dense reinforcement learning for safety validation of autonomous vehicles, Nature 615 (7953) (2023) 620–627.

[2] C. Badue, R. Guidolini, R.V. Carneiro, P. Azevedo, V.B. Cardoso, A. Forechi, L. Jesus, R. Berriel, T.M. Paixao, F. Mutz, et al., Self-driving cars: A survey, Expert Syst. Appl. 165 (2021) 113816.

[3] R.R. Negenborn, F. Goerlandt, T.A. Johansen, P. Slaets, O.A. Valdez Banda, T. Vanelslander, N.P. Ventikos, Autonomous ships are on the horizon: here's what we need to know, Nature 615 (7950) (2023) 30–33.

[4] M. Ribeiro, J. Ellerbroek, J. Hoekstra, Review of conflict resolution methods for manned and unmanned aviation, Aerospace 7 (6) (2020) 79.

[5] J.K. Kuchar, L.C. Yang, A review of conflict detection and resolution modeling methods, IEEE Trans. Intell. Transp. Syst. 1 (2000) 179–189.

[6] Y. Huang, L. Chen, P. Chen, R.R. Negenborn, P. Van Gelder, Ship collision avoidance methods: State-of-the-art, Saf. Sci. 121 (2020) 451–473.

[7] A. Sauer, N. Savinov, A. Geiger, Conditional affordance learning for driving in urban environments, in: Conference on Robot Learning, PMLR, 2018, pp. 237–252.

[8] T.I. Fossen, Handbook of Marine Craft Hydrodynamics and Motion Control, 2nd Edition, John Wiley & Sons, 2021.

[9] Z. Liu, Y. Zhang, X. Yu, C. Yuan, Unmanned surface vehicles: An overview of developments and challenges, Annu. Rev. Control 41 (2016) 71–93.

[10] R. Kandepu, B. Foss, L. Imsland, Applying the unscented Kalman filter for nonlinear state estimation, J. Process Control 18 (7–8) (2008) 753–768.

[11] P. Marin-Plaza, A. Hussein, D. Martin, A.d.l. Escalera, Global and local path planning study in a ROS-based research platform for autonomous vehicles, J. Adv. Transp. 2018 (2018).

[12] A. Vagale, R. Oucheikh, R.T. Bye, O.L. Osen, T.I. Fossen, Path planning and collision avoidance for autonomous surface vehicles I: A review, J. Mar. Sci. Technol. 26 (2021) 1292–1306.

[13] D. Falanga, K. Kleber, D. Scaramuzza, Dynamic obstacle avoidance for quadrotors with event cameras, Science Robotics 5 (40) (2020) eaaz9712.

[14] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, The MIT Press, Cambridge, 2018.

[15] O. Vinyals, I. Babuschkin, W.M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D.H. Choi, R. Powell, T. Ewalds, P. Georgiev, et al., Grandmaster level in StarCraft II using multi-agent reinforcement learning, Nature 575 (7782) (2019) 350–354.

[16] M.G. Bellemare, S. Candido, P.S. Castro, J. Gong, M.C. Machado, S. Moitra, S.S. Ponda, Z. Wang, Autonomous navigation of stratospheric balloons using reinforcement learning, Nature 588 (7836) (2020) 77–82.

[17] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F.J. R Ruiz, J. Schrittwieser, G. Swirszcz, et al., Discovering faster matrix multiplication algorithms with reinforcement learning, Nature 610 (7930) (2022) 47–53.

[18] X. Chen, L. Yao, J. McAuley, G. Zhou, X. Wang, Deep reinforcement learning in recommender systems: A survey and new perspectives, Knowl.-Based Syst. 264 (2023) 110335.

[19] Y. Yun, H. Dai, R. An, Y. Zhang, X. Shang, Doubly constrained offline reinforcement learning for learning path recommendation, Knowl.-Based Syst. 284 (2024) 111242.

[20] S.S. Mousavi, M. Schukat, E. Howley, Deep reinforcement learning: an overview, in: Proceedings of SAI Intelligent Systems Conference, Springer, 2016, pp. 426–440.

[21] J. Suárez-Varela, A. Mestres, J. Yu, L. Kuang, H. Feng, P. Barlet-Ros, A. Cabellos-Aparicio, Feature engineering for deep reinforcement learning based routing, in: International Conference on Communications, IEEE, 2019, pp. 1–6.

[22] Y. Li, Reinforcement learning applications, 2019, arXiv preprint arXiv:1908.06973.

[23] M. Everett, Y.F. Chen, J.P. How, Motion planning among dynamic, decision-making agents with deep reinforcement learning, in: International Conference on Intelligent Robots and Systems, IEEE, 2018, pp. 3052–3059.

[24] M. Brittain, P. Wei, Scalable autonomous separation assurance with heterogeneous multi-agent reinforcement learning, IEEE Trans. Autom. Sci. Eng. 19 (2022) 2837–2848.

[25] M. Everett, Y.F. Chen, J.P. How, Collision avoidance in pedestrian-rich environments with deep reinforcement learning, IEEE Access 9 (2021) 10357–10377.

[26] International Maritime Organization, COLREG: Convention on the international regulations for preventing collisions at sea, 1972.

[27] D.H. Chun, M.I. Roh, H.W. Lee, J. Ha, D. Yu, Deep reinforcement learning-based collision avoidance for an autonomous ship, Ocean Eng. 234 (2021) 109216.

[28] X. Xu, Y. Lu, G. Liu, P. Cai, W. Zhang, COLREGs-abiding hybrid collision avoidance algorithm based on deep reinforcement learning for USVs, Ocean Eng. 247 (2022) 110749.

[29] M. Waltz, N. Paulig, O. Okhrin, 2-level reinforcement learning for ships on inland waterways, 2023, arXiv preprint arXiv:2307.16769.

[30] A.S. Lenart, Collision threat parameters for a new radar display and plot technique, J. Navig. 36 (3) (1983) 404–410.

[31] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.

[32] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.

[33] M.A. Lee, Y. Zhu, K. Srinivasan, P. Shah, S. Savarese, L. Fei-Fei, A. Garg, J. Bohg, Making sense of vision and touch: Self-supervised learning of multimodal representations for contact-rich tasks, in: International Conference on Robotics and Automation, IEEE, 2019, pp. 8943–8950.

[34] J. Reif, M. Sharir, Motion planning in the presence of moving obstacles, J. ACM 41 (4) (1994) 764–790.

[35] J.C. Latombe, Robot Motion Planning, vol. 124, Springer Science & Business Media, 2012.

[36] O. Khatib, Real-time obstacle avoidance for manipulators and mobile robots, Int. J. Robotics Res. 5 (1) (1986) 90–98.

[37] J. Borenstein, Y. Koren, et al., The vector field histogram-fast obstacle avoidance for mobile robots, IEEE Trans. Robot. Autom. 7 (3) (1991) 278–288.

[38] D. Fox, W. Burgard, S. Thrun, The dynamic window approach to collision avoidance, IEEE Robot. Autom. Mag. 4 (1) (1997) 23–33.

[39] R. Simmons, The curvature-velocity method for local obstacle avoidance, in: International Conference on Robotics and Automation, vol. 4, IEEE, 1996, pp. 3375–3382.

[40] P. Fiorini, Z. Shiller, Motion planning in dynamic environments using velocity obstacles, Int. J. Robotics Res. 17 (7) (1998) 760–772.

[41] M. Dorigo, L.M. Gambardella, Ant colonies for the travelling salesman problem, Biosystems 43 (2) (1997) 73–81.

[42] S.M. LaValle, J.J. Kuffner, B. Donald, et al., Rapidly-exploring random trees: Progress and prospects, Algorithmic Comput. Robotics: New Directions 5 (2001) 293–308.

[43] J. Kennedy, R. Eberhart, Particle swarm optimization, in: International Conference on Neural Networks, vol. 4, IEEE, 1995, pp. 1942–1948.

[44] H. Lyu, Y. Yin, COLREGS-constrained real-time path planning for autonomous ships using modified artificial potential fields, J. Navig. 72 (3) (2019) 588–608.

[45] W. Liu, K. Qiu, X. Yang, R. Wang, Z. Xiang, Y. Wang, W. Xu, COLREGS-based collision avoidance algorithm for unmanned surface vehicles using modified artificial potential fields, Phys. Commun. 57 (2023) 101980.

[46] Y. Kuwata, M.T. Wolf, D. Zarzhitsky, T.L. Huntsberger, Safe maritime autonomous navigation with COLREGS, using velocity obstacles, IEEE J. Ocean. Eng. 39 (1) (2013) 110–119.

[47] S. Hu, S. Tian, J. Zhao, R. Shen, Path planning of an unmanned surface vessel based on the improved A-star and dynamic window method, J. Mar. Sci. Eng. 11 (5) (2023) 1060.

[48] C.Y. Tan, S. Huang, K.K. Tan, R.S.H. Teo, Three dimensional collision avoidance for multi unmanned aerial vehicles using velocity obstacle, J. Intell. Robot. Syst. 97 (1) (2020) 227–248.

[49] R. Polvara, S. Sharma, J. Wan, A. Manning, R. Sutton, Obstacle avoidance approaches for autonomous navigation of unmanned surface vehicles, J. Navig. 71 (1) (2018) 241–256.

[50] A. Pandey, S. Pandey, D. Parhi, Mobile robot navigation and obstacle avoidance techniques: A review, Int. Robotics Autom. J. 2 (3) (2017) 96–105.

[51] B. Patle, A. Pandey, D. Parhi, A. Jagadeesh, et al., A review: On path planning strategies for navigation of mobile robot, Def. Technol. 15 (4) (2019) 582–606.

[52] P. Zhao, Y. Liu, Physics informed deep reinforcement learning for aircraft conflict resolution, IEEE Trans. Intell. Transp. Syst. 23 (7) (2021) 8288–8301.

[53] C. Wang, J. Wang, Y. Shen, X. Zhang, Autonomous navigation of UAVs in large-scale complex environments: A deep reinforcement learning approach, IEEE Trans. Veh. Technol. 68 (3) (2019) 2124–2136.

[54] R. Isufaj, M. Omeri, M.A. Piera, Multi-UAV conflict resolution with graph convolutional reinforcement learning, Appl. Sci. 12 (2) (2022) 610.

[55] M. Waltz, O. Okhrin, Spatial–temporal recurrent reinforcement learning for autonomous ships, Neural Netw. 165 (2023) 634–653.

[56] A. Heiberg, T.N. Larsen, E. Meyer, A. Rasheed, O. San, D. Varagnolo, Risk-based implementation of COLREGs for autonomous surface vehicles using deep reinforcement learning, Neural Netw. 152 (2022) 17–33.

[57] H. Shen, H. Hashimoto, A. Matsuda, Y. Taniguchi, D. Terada, C. Guo, Automatic collision avoidance of multiple ships based on deep Q-learning, Appl. Ocean Res. 86 (2019) 268–288.

[58] M. Duguleana, G. Mogan, Neural networks based reinforcement learning for mobile robots obstacle avoidance, Expert Syst. Appl. 62 (2016) 104–115.

[59] C.J. Watkins, P. Dayan, Q-learning, Mach. Learn. 8 (3) (1992) 279–292.

[60] X. Xue, Z. Li, D. Zhang, Y. Yan, A deep reinforcement learning method for mobile robot collision avoidance based on double DQN, in: International Symposium on Industrial Electronics, IEEE, 2019, pp. 2131–2136.

[61] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, in: AAAI Conference on Artificial Intelligence, vol. 30, 2016.

[62] L. Jiang, L. An, X. Zhang, C. Wang, X. Wang, A human-like collision avoidance method for autonomous ship with attention-based deep reinforcement learning, Ocean Eng. 264 (2022) 112378.

[63] T. Xu, S. Zhang, Z. Jiang, Z. Liu, H. Cheng, Collision avoidance of high-speed obstacles for mobile robots via maximum-speed aware velocity obstacle method, IEEE Access 8 (2020) 138493–138507.

[64] J. Roghair, A. Niaraki, K. Ko, A. Jannesari, A vision based deep reinforcement learning algorithm for UAV obstacle avoidance, in: Intelligent Systems and Applications: Intelligent Systems Conference, Springer, 2022, pp. 115–128.

[65] P. Chen, Y. Huang, J. Mou, P. Van Gelder, Probabilistic risk analysis for ship-ship collision: State-of-the-art, Saf. Sci. 117 (2019) 108–122.

[66] U. Ozturk, K. Cicek, Individual collision risk assessment in ship navigation: A systematic literature review, Ocean Eng. 180 (2019) 130–143.

[67] R.A. Paielli, H. Erzberger, Conflict probability estimation for free flight, J. Guid. Control Dyn. 20 (3) (1997) 588–596.

[68] I. Hwang, C.E. Seah, Intent-based probabilistic conflict detection for the next generation air transportation system, Proc. IEEE 96 (12) (2008) 2040–2059.

[69] L. Yang, J.H. Yang, J. Kuchar, E. Feron, A real-time Monte Carlo implementation for computing probability of conflict, in: AIAA Guidance, Navigation, and Control Conference and Exhibit, 2004, p. 4876.

[70] N. Zhang, H. Liu, B.F. Ng, K.H. Low, Collision probability between intruding drone and commercial aircraft in airport restricted area based on collision-course trajectory planning, Transp. Res. C 120 (2020) 102736.

[71] Y. Zou, H. Zhang, G. Zhong, H. Liu, D. Feng, Collision probability estimation for small unmanned aircraft systems, Reliab. Eng. Syst. Saf. 213 (2021) 107619.

[72] M. Mitici, H.A. Blom, Mathematical models for air traffic conflict and collision probability estimation, IEEE Trans. Intell. Transp. Syst. 20 (3) (2018) 1052–1068.

[73] Y. Fujii, K. Tanaka, Traffic capacity, J. Navig. 24 (4) (1971) 543–552.

[74] M.G. Hansen, T.K. Jensen, T. Lehn-Schiøler, K. Melchild, F.M. Rasmussen, F. Ennemark, Empirical ship domain based on AIS data, J. Navig. 66 (6) (2013) 931–940.

[75] R. Szlapczynski, J. Szlapczynska, Review of ship safety domains: Models and applications, Ocean Eng. 145 (2017) 277–289.

[76] J.M. Mou, C. Van Der Tak, H. Ligteringen, Study on collision avoidance in busy waterways by using AIS data, Ocean Eng. 37 (5–6) (2010) 483–490.

[77] A.K. Debnath, H.C. Chin, Navigational traffic conflict technique: A proactive approach to quantitative measurement of collision risks in port waters, J. Navig. 63 (1) (2010) 137–152.

[78] K. Zheng, Y. Chen, Y. Jiang, S. Qiao, A SVM based ship collision risk assessment algorithm, Ocean Eng. 202 (2020) 107062.

[79] T. Tengesdal, T.A. Johansen, E.F. Brekke, Ship collision avoidance utilizing the cross-entropy method for collision risk assessment, IEEE Trans. Intell. Transp. Syst. 23 (8) (2021) 11148–11161.

[80] D.P. Kroese, S. Porotsky, R.Y. Rubinstein, The cross-entropy method for continuous multi-extremal optimization, Methodol. Comput. Appl. Probab. 8 (2006) 383–407.

[81] T. Tengesdal, T.A. Johansen, E. Brekke, Risk-based autonomous maritime collision avoidance considering obstacle intentions, in: International Conference on Information Fusion, IEEE, 2020, pp. 1–8.

[82] Q. Yu, A. Teixeira, K. Liu, C.G. Soares, Framework and application of multi-criteria ship collision risk assessment, Ocean Eng. 250 (2022) 111006.

[83] R.S. Tsay, Analysis of Financial Time Series, John Wiley & Sons, New Jersey, 2010.

[84] X. Zhou, H. Liu, F. Pourpanah, T. Zeng, X. Wang, A survey on epistemic (model) uncertainty in supervised learning: Recent advances and applications, Neurocomputing 489 (2021) 449–465.

[85] A.B. Nassif, I. Shahin, I. Attili, M. Azzeh, K. Shaalan, Speech recognition using deep neural networks: A systematic review, IEEE Access 7 (2019) 19143–19165.

[86] A. Dhillon, G.K. Verma, Convolutional neural network: A review of models, methodologies and applications to object detection, Prog. Artif. Intell. 9 (2) (2020) 85–112.

[87] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.

[88] M.L. Puterman, Markov Decision Processes: Discrete Stochastic Dynamic Programming, John Wiley & Sons, 1994.

[89] L.P. Kaelbling, M.L. Littman, A.R. Cassandra, Planning and acting in partially observable stochastic domains, Artificial Intelligence 101 (1–2) (1998) 99–134.

[90] N. Heess, J.J. Hunt, T.P. Lillicrap, D. Silver, Memory-based control with recurrent neural networks, 2015, arXiv preprint arXiv:1512.04455.

[91] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.

[92] T. Ni, B. Eysenbach, R. Salakhutdinov, Recurrent model-free RL is a strong baseline for many POMDPs, 2021, arXiv preprint arXiv:2110.05038.

[93] L. Meng, R. Gorbet, D. Kulić, Memory-based deep reinforcement learning for pomdps, in: International Conference on Intelligent Robots and Systems, IEEE, 2021, pp. 5619–5626.

[94] S. Fujimoto, H. Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: International Conference on Machine Learning, PMLR, 2018, pp. 1587–1596.

[95] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International Conference on Machine Learning, PMLR, 2018, pp. 1861–1870.

[96] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al., Soft actor-critic algorithms and applications, 2018, arXiv preprint arXiv:1812.05905.

[97] F. Hart, O. Okhrin, Enhanced method for reinforcement learning based dynamic obstacle avoidance by assessment of collision risk, Neurocomputing 568 (2024) 127097.

[98] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al., A general reinforcement learning algorithm that masters chess, shogi, and go through self-play, Science 362 (6419) (2018) 1140–1144.

[99] D. Yang, L. Wu, S. Wang, H. Jia, K.X. Li, How big data enriches maritime research–A critical review of Automatic Identification System (AIS) data applications, Transp. Rev. 39 (6) (2019) 755–773.

[100] International Maritime Organization, SOLAS: International convention for the safety of life at sea, 1974.

[101] P. Last, C. Bahlke, M. Hering-Bertram, L. Linsen, Comprehensive analysis of automatic identification system (AIS) data in regard to vessel movement prediction, J. Navig. 67 (5) (2014) 791–809.

[102] N. Paulig, O. Okhrin, An open-source framework for data-driven trajectory extraction from AIS data–the α-method, 2024, arXiv preprint arXiv:2407.04402.

[103] A.L. Maas, A.Y. Hannun, A.Y. Ng, et al., Rectifier nonlinearities improve neural network acoustic models, in: International Conference on Machine Learning, vol. 30, (1) Atlanta, GA, 2013, p. 3.

[104] M. Akdağ, T.I. Fossen, T.A. Johansen, Collaborative collision avoidance for autonomous ships using informed scenario-based model predictive control, IFAC-PapersOnLine 55 (31) (2022) 249–256.

[105] Y.F. Chen, M. Liu, M. Everett, J.P. How, Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning, in: International Conference on Robotics and Automation, IEEE, 2017, pp. 285–292.

[106] N. Lee, W. Choi, P. Vernaza, C.B. Choy, P.H. Torr, M. Chandraker, Desire: Distant future prediction in dynamic scenes with interacting agents, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 336–345.

[107] Y. Chai, B. Sapp, M. Bansal, D. Anguelov, MultiPath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction, in: Conference on Robot Learning, PMLR, 2020, pp. 86–99.

[108] M. Treiber, A. Kesting, Traffic Flow Dynamics: Data, Models and Simulation, Springer-Verlag Berlin Heidelberg, 2013.

[109] E. Meyer, H. Robinson, A. Rasheed, O. San, Taming an autonomous surface vehicle for path following and collision avoidance using deep reinforcement learning, IEEE Access 8 (2020) 41466–41481.